

Framework Profissional de Automação de Testes

Construindo uma Arquitetura Enterprise do Zero

Este material acompanha o projeto de referencia. Cada parte corresponde a uma etapa da construção do framework, desde a fundacao ate a integração continua.

Sumario

- [Parte 1 — Visao Geral e Arquitetura](#)
- [1.1 O que e automação de testes e por que automatizar](#)
- [1.2 O que e um framework](#)
- [1.3 Piramide de testes](#)
- [1.4 Arquitetura em camadas](#)
- [1.5 Fluxo completo de execução](#)
- [1.6 Responsabilidade de cada camada](#)
- [1.7 Decisoões tecnológicas e justificativas](#)
- [1.8 Estrutura completa de diretórios](#)
- [1.9 Pre-requisitos](#)
- [Parte 2 — Construção do Framework UI](#)
- [2.1 Criação do projeto Maven](#)
- [2.2 DriverFactory](#)
- [2.3 DriverManager](#)
- [2.4 BasePage](#)
- [2.5 LoginPage](#)
- [2.6 Cucumber — Features em portugues](#)
- [2.7 TestRunner](#)
- [2.8 LoginSteps](#)
- [2.9 UiHooks](#)
- [2.10 Evidências — Screenshots automaticas](#)
- [Parte 3 — Construção do Framework API](#)
- [3.1 REST Assured — Introducao](#)
- [3.2 RestClient](#)
- [3.3 Padrão Client-Service](#)
- [3.4 PostService](#)
- [3.5 PostRequest — Modelo POJO](#)
- [3.6 PostBuilder — Builder Pattern com Faker](#)
- [3.7 JsonUtils](#)
- [3.8 Payloads externalizados](#)
- [3.9 JSON Schema Validation](#)
- [3.10 Feature de API em portugues](#)

- [3.11 PostSteps](#)
 - [3.12 TestData vs Payload](#)
 - [Parte 4 — Engenharia do Framework](#)
 - [4.1 PicoContainer — Injecao de dependência](#)
 - [4.2 ConfigReader](#)
 - [4.3 Environment](#)
 - [4.4 Configuração por ambiente](#)
 - [4.5 Logging corporativo](#)
 - [4.6 FrameworkException](#)
 - [4.7 Screenshots configuraveis](#)
 - [4.8 Allure Report](#)
 - [4.9 Estratégia de Tags](#)
 - [4.10 Organização e nomenclatura](#)
 - [Parte 5 — Integração e Manutenção](#)
 - [5.1 GitHub Actions — Pipeline completo](#)
 - [5.2 Como adicionar um novo teste UI](#)
 - [5.3 Como adicionar um novo teste API](#)
 - [5.4 Padroes de Code Review](#)
 - [5.5 Checklist para iniciar novo projeto](#)
 - [5.6 Troubleshooting](#)
 - [5.7 FAQ](#)
 - [5.8 Glossario](#)
 - [5.9 Comandos rapidos](#)
 - [5.10 Como evoluir a arquitetura](#)
-

Parte 1 — Visao Geral e Arquitetura

1.1 O que e automação de testes e por que automatizar

Automação de testes é a pratica de usar software para executar testes que seriam feitos manualmente. Em vez de um QA clicar em cada campo, preencher formularios e verificar resultados toda vez que o sistema muda, um script faz isso de forma repetivel, rapida e confiável.

A justificativa economica e clara: um teste manual que leva 10 minutos, executado 50 vezes por sprint, consome mais de 8 horas de trabalho humano. Automatizado, ele roda em segundos a cada commit. O investimento inicial se paga em poucas semanas.

O ciclo de vida de um bug:

Quanto mais tarde um bug e encontrado, mais caro ele e para corrigir. A automação atua na camada de prevencao — detecta regressoes ANTES de chegarem a produção.

Custo de correcao por fase:		
Desenvolvimento:	1x	(barato)
Testes manuais:	5x	
QA formal:	10x	
Homologacao:	20x	
Producao:	100x	(caro + dano reputacional)

A automação de testes opera na fase de "Testes" — encontra o bug quando custa 5x, não quando custa 100x.

Quando NAO automatizar:

Nem tudo deve ser automatizado. Automatize quando: - O teste sera executado mais de 5 vezes - O fluxo e estavel (não muda toda sprint) - O resultado e deterministico (mesma entrada = mesma saida) - Ha ROI positivo (custo de automação < custo de execução manual)

Não automatize quando: - O fluxo muda constantemente (tela em desenvolvimento ativo) - E um teste exploratorio (sem roteiro definido) - E um teste de usabilidade (requer percepcao humana) - O sistema não tem interface estavel (prototipos)

Beneficios concretos:

Aspecto	Manual	Automatizado
Velocidade de execução	10-30 min por fluxo	5-30 seg por fluxo
Consistencia	Sujeito a erro humano	Execução identica sempre

Feedback	Apos horas/dias	Em minutos (CI/CD)
Cobertura de regressao	Parcial (tempo limitado)	Completa (todas as suites)
Custo por execução	Alto (hora-homem)	Praticamente zero
Execução noturna	Inviavel	Trivial

Dica: Automatizar não significa eliminar testes manuais. Testes exploratorios, de usabilidade e cenários ad-hoc continuam sendo manuais. Automatize o que e repetitivo e regressivo.

1.2 O que e um framework

Ha uma diferenca critica entre "ter scripts de teste" e "ter um framework de automação". Scripts soltos sao arquivos Java que abrem o navegador, clicam em elementos e verificam resultados — sem padrão, sem reuso, sem estrutura.

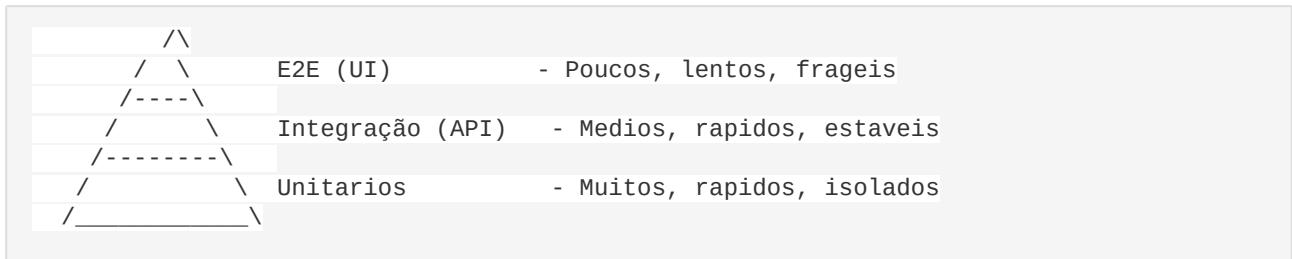
Um framework é uma arquitetura organizada que define: - **Onde** colocar cada tipo de código (separação de responsabilidades) - **Como** elementos sao localizados e interagidos (Page Objects) - **Como** dados de teste sao gerenciados (configuracoes por ambiente) - **Como** os testes sao executados e reportados (Runner + Reports) - **Como** novos testes sao adicionados (padrão replicavel)

Característica	Scripts Soltos	Framework
Manutenção	Alterar em N arquivos	Alterar em 1 lugar
Reuso de código	Copiar e colar	Herança + composicao
Onboarding de novos QAs	Dias	Horas
Escala para +100 testes	Caos	Organizado
Integração com CI/CD	Dificil	Nativa
Reports profissionais	Inexistente	Allure/Cucumber

Dica: Se você precisa mais de 1 hora para explicar como adicionar um novo teste, seu framework precisa de refatoracao.

1.3 Piramide de testes

A piramide de testes (Martin Fowler / Mike Cohn) define a proporcao ideal de testes em um sistema:



Neste framework, cobrimos as duas camadas superiores: - **Testes de UI (E2E):** Validam fluxos completos do usuario via navegador (Selenium) - **Testes de API (Integração):** Validam contratos, status codes e regras de negocio via HTTP (REST Assured)

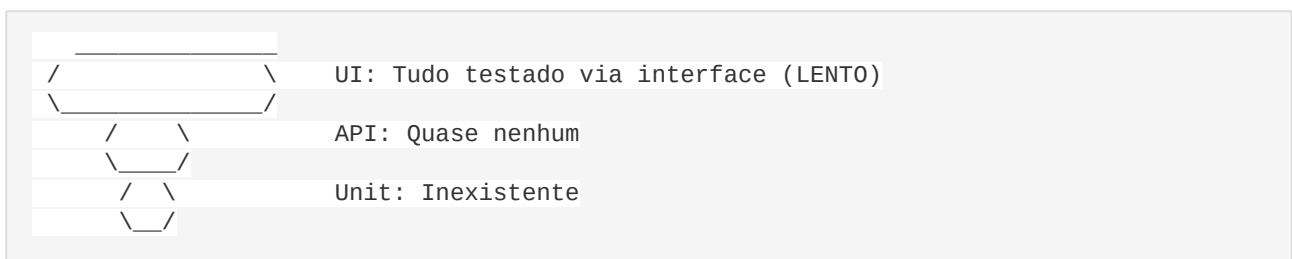
A estratégia é: manter poucos testes UI nos caminhos criticos (login, checkout) e muitos testes API para validacoes de dados e regras.

Características de cada camada:

Camada	Quantidade	Velocidade	Estabilidade	Manutenção	Confiança
Unitarios	Centenas	Milissegundos	Muito alta	Baixa	Isolada
API	Dezenas	0.5-3 segundos	Alta	Media	Integração
UI	Poucos	5-30 segundos	Media-baixa	Alta	E2E

Distribuicao recomendada: - 70% unitarios (responsabilidade do time de desenvolvimento) - 20% API (responsabilidade compartilhada QA + DEV) - 10% UI (responsabilidade principal do QA)

Anti-padrão: Piramide invertida (Ice Cream Cone)



Se sua organização tem mais testes UI do que API e unit, você está no anti-padrão "cone de sorvete". Migre gradualmente: para cada novo teste UI, pergunte "isso poderia ser um teste de API?".

Dica: Cada teste de UI que pode ser substituído por um teste de API deve ser. Testes de API são 10x mais rápidos e 5x mais estáveis.

1.4 Arquitetura em camadas

O framework segue uma arquitetura em camadas, onde cada camada tem responsabilidade única e se comunica apenas com a camada imediatamente abaixo.

[Diagrama Mermaid — visualize no arquivo .md]

Cada camada é isolada e testável independentemente. Se amanhã o Selenium for substituído por Playwright, apenas a camada de infraestrutura muda — os Steps e Features permanecem intactos.

Dica: A regra de ouro da arquitetura em camadas: uma camada **NUNCA** deve depender de uma camada acima dela. Steps conhecem Pages, mas Pages nunca importam Steps.

1.5 Fluxo completo de execução

Quando você executa `mvn test`, uma cadeia precisa de eventos acontece. Entender esse fluxo é fundamental para debugar problemas.

[Diagrama Mermaid — visualize no arquivo .md]

O ponto chave é que o Cucumber controla o ciclo de vida. Ele lê os `.feature`, mapeia cada frase para um método Java (step definition), e gerencia os hooks de setup/teardown.

Dica: Se um teste falha no `@Before`, o cenário inteiro é marcado como falho. Sempre verifique os logs de hook primeiro ao investigar falhas.

1.6 Responsabilidade de cada camada

Camada	Responsabilidade	Exemplos de Classes
Especificação	Descrever comportamento em linguagem natural	<code>login.feature</code> , <code>posts.feature</code>

Orquestração	Configurar execução e ciclo de vida	TestRunner, UiHooks, ApiHooks
Steps	Traduzir Gherkin para ações Java	LoginSteps, PostSteps
Logica (UI)	Encapsular interações com páginas	BasePage, LoginPage
Logica (API)	Encapsular regras de negócio HTTP	PostService, PostBuilder
Infraestrutura	Gerenciar conexões e configurações	DriverFactory, RestClient, Environment
Utilitários	Funções transversais de suporte	LogUtils, ScreenshotUtils, JsonUtils
Modelos	Representar dados estruturados	PostRequest
Exceções	Erros semânticos do framework	FrameworkException

Dica: Se uma classe não se encaixa claramente em nenhuma camada, provavelmente ela está fazendo coisas demais e deve ser dividida.

1.7 Decisões tecnológicas e justificativas

Tecnologia	Versão	Justificativa
Java	8	Compatibilidade máxima com ambientes corporativos; LTS
Maven	3.x	Gerenciador de build padrão; integração nativa com CI
Selenium WebDriver	3.141.59	Última versão compatível com Java 8; estável em produção
Cucumber	7.18.0	BDD em português; integração madura com JUnit

JUnit 4	4.13.2	Runner padrão para Cucumber; compatível com Surefire
REST Assured	4.5.1	Ultima versão Java 8; API fluente given/when/then
PicoContainer	7.18.0	DI leve por cenário; zero configuração
Allure Report	2.24.0	Relatórios visuais ricos; historico de execucoes
SLF4J + Logback	1.7.36 / 1.2.12	Logging desacoplado; configuravel por ambiente
JavaFaker	1.0.2	Dados dinâmicos; evita dados fixos em testes
AspectJ	1.9.19	Necessario para integração Allure (weaving)
GitHub Actions	-	CI/CD gratuito; integração nativa com repositorio

Dica: A escolha do Java 8 e estrategica: a maioria dos ambientes corporativos brasileiros ainda roda Java 8 em produção. Seu framework de testes deve ser compatível com o ambiente onde sera executado.

1.8 Estrutura completa de diretórios

```
selenium-cucumber-project/
├── .github/
│   └── workflows/
│       └── testes.yml # Pipeline CI/CD
├── src/
│   └── test/
│       └── java/
│           ├── api/
│           │   ├── builders/
│           │   │   ├── PostBuilder.java # Builder Pattern + Faker
│           │   └── clients/
│           │       ├── RestClient.java # Cliente HTTP corporativo
│           │       └── models/
│           │           ├── PostRequest.java # POJO de request
│           │           └── services/
│           │               └── PostService.java # Logica de negocio API
```

```

■      ■      ■■■ config/
■      ■      ■      ■■■ ConfigReader.java          # Leitura de .properties
■      ■      ■      ■■■ Environment.java           # Gerenciamento de ambientes
■      ■      ■■■ drivers/
■      ■      ■      ■■■ DriverFactory.java          # Criação do navegador
■      ■      ■      ■■■ DriverManager.java          # ThreadLocal para paralelo
■      ■      ■■■ exceptions/
■      ■      ■      ■■■ FrameworkException.java     # Exceção customizada
■      ■      ■■■ hooks/
■      ■      ■      ■■■ ApiHooks.java               # Hooks para @api
■      ■      ■      ■■■ UiHooks.java                # Hooks para @ui
■      ■      ■■■ pages/
■      ■      ■      ■■■ base/
■      ■      ■      ■      ■■■ BasePage.java         # Classe abstrata base
■      ■      ■      ■      ■■■ login/
■      ■      ■      ■      ■■■ LoginPage.java        # Page Object do login
■      ■      ■■■ runners/
■      ■      ■      ■■■ TestRunner.java              # Ponto de entrada
■      ■      ■■■ steps/
■      ■      ■      ■■■ api/
■      ■      ■      ■      ■■■ PostSteps.java        # Steps de API
■      ■      ■      ■      ■■■ ui/
■      ■      ■      ■      ■■■ LoginSteps.java       # Steps de UI
■      ■      ■■■ utils/
■      ■      ■      ■■■ JsonUtils.java               # Carregamento de JSON
■      ■      ■      ■■■ LogUtils.java                # Logging corporativo
■      ■      ■      ■■■ ScreenshotUtils.java         # Captura de tela
■      ■■■ resources/
■      ■      ■■■ environments/
■      ■      ■      ■■■ dev.properties               # Configuração DEV
■      ■      ■      ■■■ hml.properties               # Configuração HML
■      ■      ■■■ features/
■      ■      ■      ■■■ api/
■      ■      ■      ■      ■■■ posts.feature          # Cenários de API
■      ■      ■      ■      ■■■ ui/
■      ■      ■      ■      ■■■ login.feature          # Cenários de UI
■      ■      ■■■ payloads/
■      ■      ■      ■■■ posts/
■      ■      ■      ■      ■■■ create-post.json       # Payload de criação
■      ■      ■      ■      ■■■ update-post.json       # Payload de atualizacao
■      ■      ■■■ schemas/
■      ■      ■      ■■■ post-schema.json             # Schema de validação
■      ■      ■      ■■■ logback.xml                  # Configuração de logging
■■■ pom.xml                                           # Configuração Maven

```

A organização segue o princípio de **coesão**: arquivos que mudam juntos ficam juntos. Todas as classes de API ficam em `api/`, todos os recursos de API ficam em subpastas correspondentes.

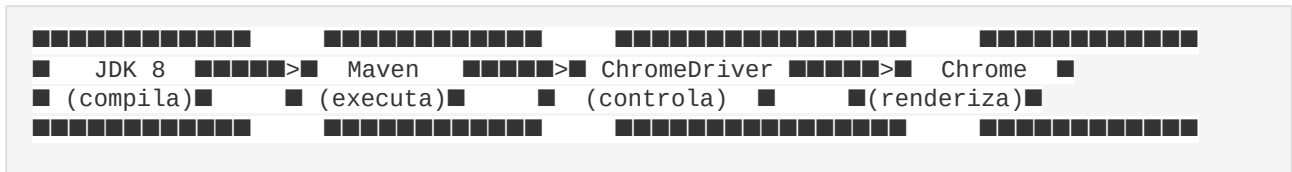
Dica: Ao criar um novo recurso (ex: Users), replique a mesma estrutura:

`api/services/UserService.java`, `api/models/UserRequest.java`,
`payloads/users/create-user.json`, `features/api/users.feature`.

1.9 Pre-requisitos

Para executar este framework localmente, você precisa de quatro componentes fundamentais. Cada um cumpre um papel específico na cadeia de execução.

Visão geral dos componentes:



- **JDK 8** compila o código Java do framework
- **Maven** gerencia dependências e executa os testes
- **ChromeDriver** é a ponte entre o Selenium e o Chrome
- **Chrome** é o navegador que renderiza as páginas

1. JDK 8 (Java Development Kit)

O JDK inclui o compilador (`javac`) e a JVM (`java`). Certifique-se de instalar o JDK (não apenas o JRE).

Verifique se já está instalado:

```
java -version
# Esperado: java version "1.8.x"

javac -version
# Esperado: javac 1.8.x
```

Caso não tenha, instale o Temurin (AdoptOpenJDK): - Windows: Baixe em <https://adoptium.net/> e escolha JDK 8 LTS - Configure `JAVA_HOME` apontando para a pasta de instalação - Adicione `%JAVA_HOME%\bin` ao `PATH` - Reinicie o terminal após configurar

Verificação pos-instalação:

```
echo %JAVA_HOME%
# Esperado: C:\Program Files\Eclipse Adoptium\jdk-8.x.x-hotspot (ou similar)
```

2. Apache Maven 3.x

O Maven gerencia dependências e o ciclo de build. Ele baixa automaticamente todas as bibliotecas declaradas no `pom.xml`.

```
mvn -version
# Esperado: Apache Maven 3.x.x
# Esperado: Java version: 1.8.x (confirma que usa o JDK correto)
```

- Windows: Baixe em <https://maven.apache.org/download.cgi>

- Extraia para `C:\maven`
- Configure `MAVEN_HOME` e adicione `%MAVEN_HOME%\bin` ao `PATH`

Primeiro uso: Na primeira execução, o Maven baixa todas as dependências (~100 MB). As execuções seguintes são rápidas porque usa cache local (`~/.m2/repository/`).

3. Google Chrome (última versão)

O framework usa Chrome como navegador padrão. Mantenha-o atualizado para evitar incompatibilidades com o ChromeDriver.

Para verificar a versão: acesse `chrome://version` na barra de endereço.

4. ChromeDriver

O ChromeDriver deve ser compatível com a versão do Chrome instalada: - Verifique a versão do Chrome em `chrome://version` - Baixe o driver correspondente em <https://chromedriver.chromium.org/downloads> - Para Chrome 115+: <https://googlechromelabs.github.io/chrome-for-testing/> - Coloque em um diretório acessível (ex: `C:\chromedriver\`) - Configure a variável de ambiente `CHROME_DRIVER_PATH` ou use o path padrão

Compatibilidade Chrome/ChromeDriver:

Chrome	ChromeDriver
120.x	chromedriver 120.x
121.x	chromedriver 121.x
125.x	chromedriver 125.x

A regra é simples: a versão MAJOR (primeiro número) deve ser idêntica.

Validação completa do ambiente:

```
java -version && mvn -version && google-chrome --version && chromedriver --version
```

Se todos os quatro comandos retornarem versões, seu ambiente está pronto.

Dica: Em ambientes CI (GitHub Actions), o Chrome e o ChromeDriver são instalados automaticamente pelo pipeline. A configuração local é necessária apenas para desenvolvimento.

Parte 2 — Construção do Framework UI

2.1 Criação do projeto Maven

O `pom.xml` é o coração de qualquer projeto Java gerenciado pelo Maven. Ele define as dependências, plugins de build e configurações de compilação. Cada dependência foi escolhida com critério de compatibilidade com Java 8 e maturidade em produção.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.automação</groupId>
  <artifactId>selenium-restassured-cucumber-github-actions</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <name>selenium-restassured-cucumber-github-actions</name>
  <description>Automação de testes Web (Selenium) e API (REST Assured) com Cucumber BDD</description>

  <properties>
    <java.version>8</java.version>
    <maven.compiler.source>${java.version}</maven.compiler.source>
    <maven.compiler.target>${java.version}</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

    <!-- Versões das dependências -->
    <!-- Selenium 3.x – última versão compatível com Java 8 -->
    <selenium.version>3.141.59</selenium.version>
    <!-- Cucumber 7.x requer Java 8+ -->
    <cucumber.version>7.18.0</cucumber.version>
    <junit.version>4.13.2</junit.version>
    <!-- REST Assured 4.x – última versão com suporte a Java 8 -->
    <rest.assured.version>4.5.1</rest.assured.version>
    <!-- Allure Report -->
    <allure.version>2.24.0</allure.version>
  </properties>

  <dependencies>

    <!-- Selenium WebDriver -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>${selenium.version}</version>
    </dependency>

    <!-- Cucumber JVM -->
    <dependency>
      <groupId>io.cucumber</groupId>
      <artifactId>cucumber-java</artifactId>
      <version>${cucumber.version}</version>
```

```

</dependency>

<!-- Cucumber + JUnit integration -->
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit</artifactId>
  <version>${cucumber.version}</version>
  <scope>test</scope>
</dependency>

<!-- JUnit -->
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>${junit.version}</version>
  <scope>test</scope>
</dependency>

<!-- REST Assured - testes de API REST -->
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>${rest.assured.version}</version>
  <scope>test</scope>
</dependency>

<!-- JSON Schema Validator - valida estrutura da resposta -->
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>json-schema-validator</artifactId>
  <version>${rest.assured.version}</version>
  <scope>test</scope>
</dependency>

<!-- Allure Cucumber Integration -->
<dependency>
  <groupId>io.qameta.allure</groupId>
  <artifactId>allure-cucumber7-jvm</artifactId>
  <version>${allure.version}</version>
  <scope>test</scope>
  <exclusions>
    <exclusion>
      <groupId>io.cucumber</groupId>
      <artifactId>gherkin</artifactId>
    </exclusion>
    <exclusion>
      <groupId>io.cucumber</groupId>
      <artifactId>messages</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<!-- AspectJ Weaver (necessário para Allure) -->
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
  <version>1.9.19</version>
  <scope>test</scope>
</dependency>

<!-- PicoContainer – Injecao de Dependência por cenário no Cucumber -->

```



```

<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-picocontainer</artifactId>
  <version>${cucumber.version}</version>
  <scope>test</scope>
</dependency>

<!-- SLF4J API – fachada de logging -->
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.36</version>
</dependency>

<!-- Logback – implementação SLF4J -->
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.2.12</version>
</dependency>

<!-- JavaFaker – geração de dados de teste dinâmicos -->
<dependency>
  <groupId>com.github.javafaker</groupId>
  <artifactId>javafaker</artifactId>
  <version>1.0.2</version>
  <scope>test</scope>
</dependency>

</dependencies>

<build>
  <plugins>

    <!-- Maven Surefire Plugin - executa os testes JUnit -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
      <configuration>
        <includes>
          <include>*/TestRunner.java</include>
        </includes>
        <argLine>
          -Djavax.net.ssl.trustStore=${user.home}/.maven-cacerts
          -Djavax.net.ssl.trustStorePassword=changeit
          -javaagent:"${settings.localRepository}/org/aspectj/aspectjweaver/1.9.19/as
        </argLine>
        <systemPropertyVariables>
          <allure.results.directory>target/allure-results</allure.results.directory>
        </systemPropertyVariables>
      </configuration>
    </plugin>

    <!-- Maven Compiler Plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.13.0</version>
      <configuration>
        <source>${java.version}</source>

```

```

                <target>${java.version}</target>
                <encoding>UTF-8</encoding>
            </configuration>
        </plugin>
    <!-- Allure Maven Plugin - gera relatório -->
    <plugin>
        <groupId>io.qameta.allure</groupId>
        <artifactId>allure-maven</artifactId>
        <version>2.12.0</version>
        <configuration>
            <reportVersion>${allure.version}</reportVersion>
            <resultsDirectory>allure-results</resultsDirectory>
        </configuration>
    </plugin>
</plugins>
</build>
</project>

```

Decisões de design do pom.xml:

- Todas as versões estão centralizadas em `<properties>` para facilitar atualizações futuras — basta alterar um número em um lugar.
- O Surefire está configurado para encontrar apenas `TestRunner.java`, garantindo que o Cucumber controle a execução (e não o JUnit direto).
- O `argLine` do Surefire injeta o AspectJ como agente JVM, necessário para que o Allure intercepte os steps automaticamente.
- As exclusões no `allure-cucumber7-jvm` evitam conflitos de versão com as bibliotecas Gherkin que já vem com o Cucumber.

Dica: Sempre defina versões como *properties*. Quando precisar atualizar o Cucumber de 7.18 para 7.19, você muda em UM lugar e todas as dependências se ajustam.

2.2 DriverFactory

A DriverFactory é responsável por criar instâncias do WebDriver. Ela encapsula toda a lógica de configuração do navegador, incluindo a detecção automática de ambiente CI para modo headless.

```

package drivers;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import utils.LogUtils;

```

```

/**
 * Cria instâncias de WebDriver.
 */
public class DriverFactory {

    private static final boolean IN_CI =
        System.getenv("CI") != null || System.getenv("JENKINS_URL") != null;

    public WebDriver create(String browser) {
        LogUtils.info("Criando driver: " + browser + (IN_CI ? " [headless]" : " [visual]"));
        switch (browser.toLowerCase()) {
            case "chrome": return createChrome();
            default: throw new IllegalArgumentException("Browser não suportado: " + browser);
        }
    }

    private WebDriver createChrome() {
        if (!IN_CI) {
            String path = System.getenv("CHROME_DRIVER_PATH") != null
                ? System.getenv("CHROME_DRIVER_PATH")
                : "C:\\chromedriver\\chromedriver-win64\\chromedriver.exe";
            System.setProperty("webdriver.chrome.driver", path);
        }

        ChromeOptions options = new ChromeOptions();
        if (IN_CI) {
            options.addArguments("--headless=new", "--no-sandbox",
                "--disable-dev-shm-usage", "--window-size=1920,1080");
        } else {
            options.addArguments("--start-maximized");
        }
        options.addArguments("--disable-notifications", "--remote-allow-origins=*");
        return new ChromeDriver(options);
    }
}

```

Decisões de design:

- A detecção de CI usa variáveis de ambiente padrão (CI para GitHub Actions, JENKINS_URL para Jenkins). Isso permite que o mesmo código rode local e no pipeline sem nenhuma alteração.
- Em CI, o Chrome roda headless com `--no-sandbox` e `--disable-dev-shm-usage` — flags obrigatórias para containers Linux.
- O `--window-size=1920,1080` garante que screenshots em CI tenham resolução consistente.
- Localmente, o path do ChromeDriver aceita variável de ambiente ou usa um padrão sensato.

Dica: Nunca faça commit de caminhos absolutos locais (ex: `C:\Users\joao\...`). Use variáveis de ambiente para paths que variam entre máquinas.

2.3 DriverManager

O DriverManager resolve um problema critico: como compartilhar a mesma instância de WebDriver entre Hooks, Steps e Pages sem acopla-los via campo estático comum. A resposta e ThreadLocal.

```
package drivers;

import org.openqa.selenium.WebDriver;

/**
 * Gerencia o WebDriver via ThreadLocal (seguro para paralelo).
 */
public class DriverManager {

    private static final ThreadLocal<WebDriver> driver = new ThreadLocal<>();

    private DriverManager() {}

    public static WebDriver getDriver() {
        return driver.get();
    }

    public static void setDriver(WebDriver webDriver) {
        driver.set(webDriver);
    }

    public static void quit() {
        WebDriver d = driver.get();
        if (d != null) {
            d.quit();
            driver.remove();
        }
    }
}
```

Decisoes de design:

- `ThreadLocal<WebDriver>` garante que cada thread tem sua propria instância do navegador. Quando o Cucumber executar cenários em paralelo (futuro), cada cenário tera seu Chrome isolado sem interferencia.
- O construtor e privado (padrão utility class) porque todos os métodos sao estáticos.
- O `quit()` faz `driver.remove()` após fechar — isso previne memory leaks em execucoes longas com muitos cenários.

Dica: *ThreadLocal e essencial para paralelismo, mas também e util em execução serial: ele garante um ponto único de acesso ao driver sem precisar passar o objeto por parametro em toda a cadeia.*

2.4 BasePage

A BasePage é a classe abstrata que serve de fundacao para todos os Page Objects. Ela encapsula as interacoes comuns com o Selenium (clicar, digitar, navegar) e configura o WebDriverWait com timeout do

ambiente.

```
package pages.base;

import config.Environment;
import org.openqa.selenium.By;
import org.openqa.selenium.TimeoutException;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import utils.LogUtils;

/**
 * Classe base para todos os Page Objects.
 */
public abstract class BasePage {

    protected final WebDriver driver;
    protected final WebDriverWait wait;

    protected BasePage(WebDriver driver) {
        this.driver = driver;
        int timeout = new Environment().getInt("timeout.explicit", 10);
        this.wait = new WebDriverWait(driver, timeout);
    }

    protected void navigate(String url) {
        LogUtils.info("Navegando: " + url);
        driver.get(url);
    }

    protected void type(By locator, String text) {
        WebElement element = wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
        element.clear();
        element.sendKeys(text);
    }

    protected void click(By locator) {
        wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
    }

    protected String getText(By locator) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator)).getText();
    }

    protected boolean urlContains(String fragment) {
        try {
            return wait.until(ExpectedConditions.urlContains(fragment));
        } catch (TimeoutException e) {
            LogUtils.warn("Timeout aguardando URL conter: " + fragment);
            return false;
        }
    }
}
```

Decisoões de design:

- A classe é `abstract` porque nunca deve ser instanciada diretamente — ela só existe para ser herdada por `Pages` concretas.
- Os métodos são `protected` (não `public`) porque são destinados apenas às subclasses, não ao mundo externo. `Steps` nunca devem chamar `type()` diretamente.
- O `timeout` vem do `Environment`, então cada ambiente pode ter `timeouts` diferentes (`DEV` = 10s, `HML` = 15s).
- Todo `type()` faz `clear()` antes de `sendKeys()` para evitar concatenação de texto em campos pré-preenchidos.
- O `urlContains()` captura `TimeoutException` e retorna `false` em vez de propagar a exceção — isso torna as `assertões` mais limpas nos `Steps`.

Dica: Se você precisa de uma interação específica que não existe na `BasePage` (ex: `drag-and-drop`), adicione-a aqui. Todas as `Pages` herdam automaticamente.

2.5 LoginPage

O `LoginPage` implementa o `Page Object Pattern`: cada página da aplicação tem uma classe Java correspondente que encapsula seus elementos e ações. O código externo (`Steps`) nunca lida com localizadores — apenas com métodos semânticos.

```
package pages.login;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import pages.base.BasePage;

/**
 * Page Object da página de Login.
 */
public class LoginPage extends BasePage {

    private final By usernameField = By.name("username");
    private final By passwordField = By.name("password");
    private final By loginButton = By.cssSelector("button[type='submit']");
    private final By errorMessage = By.cssSelector(".oxd-alert-content-text");

    public LoginPage(WebDriver driver) {
        super(driver);
    }

    public void open(String url) {
        navigate(url);
    }

    public void fillUsername(String username) {
        type(usernameField, username);
    }
}
```

```

    public void fillPassword(String password) {
        type(passwordField, password);
    }

    public void clickLogin() {
        click(loginButton);
    }

    public String getErrorMessage() {
        return getText(errorMessage);
    }

    public boolean isOnDashboard() {
        return urlContains("/dashboard");
    }
}

```

Decisões de design:

- Localizadores são `private final` — eles pertencem exclusivamente a esta Page e nunca mudam após construção.
- Métodos têm nomes semânticos (`fillUsername`, `clickLogin`) em vez de técnicos (`typeInField1`, `clickButton`). Isso torna os Steps legíveis como documentação.
- A Page recebe o driver no construtor (injeção de dependência) em vez de buscar de um singleton. Isso facilita testes unitários da própria Page.
- Não há lógica de negócio na Page: ela não decide se o login foi bem-sucedido — apenas expõe `isOnDashboard()` para o Step decidir.

Dica: Um Page Object nunca deve conter asserções (Assert). A responsabilidade de verificar é do Step. A Page apenas informa o estado atual.

2.6 Cucumber — Features em português

O Cucumber permite escrever especificações executáveis em linguagem natural. Com a diretiva `# language: pt`, todas as keywords ficam em português, tornando os cenários acessíveis para analistas de negócio e POs.

```

# language: pt
@ui
Funcionalidade: Login no sistema
    Como um usuário registrado
    Quero fazer login na aplicação
    Para acessar as funcionalidades do sistema

Contexto:
    Dado que estou na página de login

```

@smoke

Cenário: Login com credenciais válidas

Quando faço login como administrador

Então devo ser redirecionado para a página inicial

Cenário: Login com senha incorreta

Quando faço login com usuário "admin" e senha incorreta

Então devo ver a mensagem de erro "Invalid credentials"

Esquema do Cenário: Login com credenciais inválidas

Quando faço login com usuário "<usuario>" e senha "<senha>"

Então devo ver a mensagem de erro "<mensagem>"

Exemplos:

	usuario		senha		mensagem	
	usuarioErrado		admin123		Invalid credentials	
	wronguser		wrongpass		Invalid credentials	

Tabela de keywords Gherkin em portugues:

Portugues	Ingles	Funcao
Funcionalidade	Feature	Agrupa cenários relacionados
Contexto	Background	Steps executados antes de cada cenário
Cenário	Scenário	Caso de teste individual
Esquema do Cenário	Scenário Outline	Template com multiplas combinacoes
Exemplos	Examples	Tabela de dados para Outline
Dado	Given	Pre-condicao (setup)
Quando	When	Acao do usuario
Entao	Then	Resultado esperado
E	And	Continuacao do step anterior

Decisoões de design:

- A tag @ui permite filtrar apenas testes de interface. A tag @smoke marca cenários criticos para execução rapida.
- O Contexto (Background) evita repetir "Dado que estou na pagina de login" em todo cenário.

- O Esquema do Cenário com Exemplos e um teste parametrizado — executa o mesmo fluxo com dados diferentes.

Dica: Features devem ser escritas na perspectiva do usuário, não do sistema. Use "Quando faço login" (usuário) em vez de "Quando o sistema valida credenciais" (técnico).

2.7 TestRunner

O TestRunner é o ponto de entrada que conecta Maven, JUnit e Cucumber. Ele define onde estão as features, onde estão os steps, e quais plugins de report serão usados.

```
package runners;

import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;
import org.junit.runner.RunWith;

/**
 * Runner principal.
 *
 * mvn test -> todos
 * mvn test -Dcucumber.filter.tags="@smoke" -> smoke
 * mvn test -Dcucumber.filter.tags="@api" -> API
 * mvn test -Dcucumber.filter.tags="@ui" -> UI
 * mvn test -Denvironment=hml -> ambiente HML
 * mvn allure:serve -> relatório
 */
@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = {"steps", "hooks"},
    plugin = {
        "pretty",
        "html:target/cucumber-reports/cucumber.html",
        "json:target/cucumber-reports/cucumber.json",
        "junit:target/cucumber-reports/cucumber.xml",
        "io.qameta.allure.cucumber7jvm.AllureCucumber7Jvm"
    },
    monochrome = true
)
public class TestRunner {
}
```

Decisões de design:

- `features` aponta para a pasta raiz — o Cucumber escaneia recursivamente todas as subpastas (`ui/`, `api/`).
- `glue` inclui tanto `steps` quanto `hooks`. Se você esquecer de incluir `hooks`, os `@Before/After` não serão executados.

- Os quatro plugins geram: saída colorida no console (`pretty`), relatório HTML nativo, JSON para integração com ferramentas, XML para JUnit report, e dados para o Allure.
- `monochrome = true` remove caracteres de escape ANSI do output — útil em logs de CI.
- A filtragem por tags é feita via linha de comando (`-Dcucumber.filter.tags`), não no Runner. Isso permite flexibilidade sem recompilar.

Dica: Nunca coloque tags fixas no `@CucumberOptions` em produção. Isso impediria a execução de todos os testes. Use `-Dcucumber.filter.tags` no comando Maven.

2.8 LoginSteps

Os Step Definitions conectam a linguagem natural do Gherkin com código Java executável. Cada frase "Dado/Quando/Então" mapeia para um método anotado. O PicoContainer injeta dependências automaticamente via construtor.

```
package steps.ui;

import config.Environment;
import drivers.DriverManager;
import io.cucumber.java.pt.Dado;
import io.cucumber.java.pt.Então;
import io.cucumber.java.pt.Quando;
import org.junit.Assert;
import pages.login.LoginPage;

/**
 * Steps de Login (UI).
 * Environment injetado via PicoContainer.
 */
public class LoginSteps {

    private final Environment env;
    private LoginPage loginPage;

    public LoginSteps(Environment env) {
        this.env = env;
    }

    @Dado("que estou na página de login")
    public void openLogin() {
        loginPage = new LoginPage(DriverManager.getDriver());
        loginPage.open(env.baseUrl);
    }

    @Quando("faço login como administrador")
    public void loginAsAdmin() {
        loginPage.fillUsername(env.get("usuario.admin"));
        loginPage.fillPassword(env.get("senha.admin"));
        loginPage.clickLogin();
    }
}
```

```

    @Quando("faço login com usuário {string} e senha {string}")
    public void loginWith(String user, String pass) {
        loginPage.fillUsername(user);
        loginPage.fillPassword(pass);
        loginPage.clickLogin();
    }

    @Quando("faço login com usuário {string} e senha incorreta")
    public void loginWithWrongPassword(String user) {
        loginPage.fillUsername(user);
        loginPage.fillPassword(env.get("senha.invalida"));
        loginPage.clickLogin();
    }

    @Então("devo ser redirecionado para a página inicial")
    public void shouldBeOnDashboard() {
        Assert.assertTrue("Não redirecionou para o dashboard", loginPage.isOnDashboard());
    }

    @Então("devo ver a mensagem de erro {string}")
    public void shouldSeeError(String expected) {
        Assert.assertEquals("Mensagem incorreta", expected, loginPage.getErrorMessage());
    }
}

```

Decisões de design:

- O `Environment` é injetado via construtor pelo `PicoContainer`. Nenhuma configuração XML ou anotação adicional é necessária — basta declarar no construtor.
- Credenciais vêm do arquivo de propriedades (`env.get("usuario.admin")`), nunca hardcoded no `Step`. Isso permite trocar credenciais por ambiente sem alterar código.
- O `LoginPage` é instanciado no `step @Dado` (não no construtor) porque o driver só existe após o Hook abrir o navegador.
- Steps usam `{string}` como expressão do Cucumber para capturar parâmetros dinâmicos do Gherkin.

Dica: Cada classe de Steps deve ser coesa — trate apenas de um assunto. `LoginSteps` cuida de login, não de cadastro. Se a feature crescer, crie novas classes de Steps.

2.9 UiHooks

Os Hooks controlam o ciclo de vida do navegador: abrir antes de cada cenário UI e fechar depois. Eles também gerenciam a captura de evidências (screenshots).

```

package hooks;

import config.Environment;

```

```

import drivers.DriverFactory;
import drivers.DriverManager;
import io.cucumber.java.After;
import io.cucumber.java.Before;
import io.cucumber.java.Scenário;
import org.openqa.selenium.WebDriver;
import utils.LogUtils;
import utils.ScreenshotUtils;

import java.util.concurrent.TimeUnit;

/**
 * Hooks para cenários @ui.
 */
public class UiHooks {

    private final Environment env;

    public UiHooks() {
        this.env = new Environment();
    }

    @Before(value = "@ui", order = 0)
    public void logScenário(Scenário cenário) {
        LogUtils.info("=== [UI] " + cenário.getName() + " ===");
    }

    @Before(value = "@ui", order = 1)
    public void openBrowser() {
        if (DriverManager.getDriver() == null) {
            String browser = env.get("browser", "chrome");
            int implicitWait = env.getInt("timeout.implicit", 10);
            int pageLoad = env.getInt("timeout.pageLoad", 30);

            DriverFactory factory = new DriverFactory();
            WebDriver driver = factory.create(browser);
            driver.manage().timeouts().implicitlyWait(implicitWait, TimeUnit.SECONDS);
            driver.manage().timeouts().pageLoadTimeout(pageLoad, TimeUnit.SECONDS);
            DriverManager.setDriver(driver);
        }
    }

    @After(value = "@ui")
    public void closeBrowser(Scenário cenário) {
        WebDriver driver = DriverManager.getDriver();
        if (driver == null) return;

        String mode = env.get("screenshot.mode", "failure_only");
        boolean shouldCapture = "always".equals(mode) || cenário.isFailed();

        if (shouldCapture) {
            byte[] screenshot = ScreenshotUtils.capture(driver);
            if (screenshot.length > 0) {
                String status = cenário.isFailed() ? "FALHA" : "SUCESSO";
                cenário.attach(screenshot, "image/png", status + " - " + cenário.getName());
                LogUtils.info("Screenshot [" + status + "]");
            }
        }

        DriverManager.quit();
        LogUtils.info("=== Navegador encerrado ===");
    }
}

```

```
}  
}
```

Ciclo de vida do cenário UI:

[Diagrama Mermaid — visualize no arquivo .md]

Decisões de design:

- `value = "@ui"` faz o hook rodar APENAS em cenários com a tag `@ui`. Cenários de API não abrem navegador.
- `order = 0` e `order = 1` garantem sequencia: primeiro loga, depois abre o browser. Ordens menores executam primeiro.
- O `if (DriverManager.getDriver() == null)` previne reabrir o navegador se ele já estiver aberto (defensivo).
- O screenshot mode é configurável por ambiente: em DEV capture apenas falhas (economiza tempo), em HML capture sempre (mais evidências).
- `cenário.attach()` anexa a screenshot diretamente ao relatório Cucumber/Allure.

Dica: O `@After` sempre executa, mesmo se o cenário falhou. Por isso ele é o local perfeito para *cleanup* (fechar browser, limpar dados). Nunca faça *cleanup* no último step.

2.10 Evidências — Screenshots automáticas

A captura de evidências é essencial para rastreabilidade. O `ScreenshotUtils` encapsula a lógica de captura, enquanto o `UiHooks` decide QUANDO capturar com base na configuração.

```
package utils;  
  
import org.openqa.selenium.OutputType;  
import org.openqa.selenium.TakesScreenshot;  
import org.openqa.selenium.WebDriver;  
  
/**  
 * Captura de screenshots.  
 */  
public class ScreenshotUtils {  
  
    private ScreenshotUtils() {}  
  
    public static byte[] capture(WebDriver driver) {  
        if (driver instanceof TakesScreenshot) {  
            return ((TakesScreenshot) driver).getScreenshotAs(OutputType.BYTES);  
        }  
        return new byte[0];  
    }  
}
```

```
}  
}
```

Modos de captura:

Modo	Configuração	Quando captura	Uso recomendado
<code>failure_only</code>	<code>screenshot.mode=failure_only</code>	Apenas quando cenário falha	DEV (economiza tempo)
<code>always</code>	<code>screenshot.mode=always</code>	Sempre (sucesso e falha)	HML (mais evidências)

A decisão de modo fica no arquivo de propriedades do ambiente. O código em UiHooks avalia:

```
String mode = env.get("screenshot.mode", "failure_only");  
boolean shouldCapture = "always".equals(mode) || cenário.isFailed();
```

A screenshot é retornada como `byte[]` e anexada ao cenário Cucumber via `cenário.attach()`. Isso garante que a imagem aparece tanto no relatório Cucumber HTML quanto no Allure Report.

Dica: Em ambientes de homologação, use `always` — isso gera evidências visuais para auditoria mesmo quando os testes passam.

Parte 3 — Construção do Framework API

3.1 REST Assured — Introducao

REST Assured é uma biblioteca Java para testar APIs RESTful de forma fluente. Ela usa a mesma estrutura Given/When/Then do BDD para construir requisicoes HTTP e validar respostas.

Conceitos fundamentais:

```
given() // Configuração (headers, body, auth)
    .baseUrl("https://api.example.com")
    .contentType(ContentType.JSON)
    .body(payload)
.when() // Acao (verbo HTTP)
    .post("/users")
.then() // Verificação (assercoes)
    .statusCode(201)
    .body("id", notNullValue());
```

Anatomia de uma chamada REST:

Uma chamada de API é composta por:

[illegible]

Nos testes, validamos: 1. **Status Code** — a API retornou o código esperado? 2. **Headers** — o Content-Type esta correto? 3. **Body** — os campos tem os valores esperados? 4. **Schema** — a estrutura da resposta esta correta?

Por que REST Assured em vez de HttpClient puro:

Aspecto	HttpClient	REST Assured
---------	------------	--------------

Sintaxe	Verbosa (20+ linhas)	Fluente (5 linhas)
Validacoes	Manuais (parse JSON + assert)	Built-in (jsonPath, schema)
Logging	Manual	Automatico (log().all())
Integração BDD	Nenhuma	Nativa (given/when/then)
Curva de aprendizado	Alta	Baixa
Serialização JSON	Manual	Automatica
Cookie handling	Manual	Automatico

Exemplo comparativo:

HttpClient (verboso):

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/posts/1"))
    .header("Accept", "application/json")
    .GET()
    .build();
HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
JSONObject json = new JSONObject(response.body());
assertEquals(200, response.statusCode());
assertEquals(1, json.getInt("id"));
```

REST Assured (fluente):

```
given()
    .baseUrl("https://api.example.com")
.when()
    .get("/posts/1")
.then()
    .statusCode(200)
    .body("id", equalTo(1));
```

O REST Assured reduz o código de teste em ~70% mantendo a mesma cobertura.

Neste framework, o REST Assured é encapsulado dentro do `RestClient` para adicionar features corporativas (logging, Allure attachments, reuso de configuração).

Dica: REST Assured foi projetado para TESTES, não para produção. Nunca use em código de aplicação — use `HttpClient` ou `OkHttp` para isso.

3.2 RestClient

O RestClient é o cliente HTTP corporativo do framework. Ele encapsula o REST Assured adicionando logging automatico, attachments para o Allure, é uma interface simplificada para os Services.

```
package api.clients;

import io.qameta.allure.Allure;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import io.restassured.specification.RequestSpecification;
import utils.LogUtils;

import static io.restassured.RestAssured.given;

/**
 * Cliente HTTP corporativo.
 * Instancia por cenário (thread-safe via PicoContainer).
 * Anexa request/response ao Allure automaticamente.
 */
public class RestClient {

    private Response response;
    private RequestSpecification request;
    private String baseUrl;
    private String lastBody;

    public RestClient() {}

    public void setBaseUrl(String baseUrl) {
        this.baseUrl = baseUrl;
        this.request = given()
            .baseUrl(baseUrl)
            .contentType(ContentType.JSON)
            .accept(ContentType.JSON);
    }

    public void addHeader(String key, String value) {
        request = request.header(key, value);
    }

    public void setBody(String body) {
        this.lastBody = body;
        request = request.body(body);
    }

    public void get(String endpoint) { execute("GET", endpoint); }
    public void post(String endpoint) { execute("POST", endpoint); }
    public void put(String endpoint) { execute("PUT", endpoint); }
    public void delete(String endpoint) { execute("DELETE", endpoint); }

    public int getStatusCode() { return response.getStatusCode(); }
    public String getContentType() { return response.getContentType(); }
    public Response getResponse() { return response; }
    public String getResponseBody() { return response.getBody().asString(); }

    private void execute(String method, String endpoint) {
        LogUtils.info(method + " " + baseUrl + endpoint);
        switch (method) {
```

```

        case "GET": response = request.when().get(endpoint).then().extract().response(); break;
        case "POST": response = request.when().post(endpoint).then().extract().response(); break;
        case "PUT": response = request.when().put(endpoint).then().extract().response(); break;
        case "DELETE": response = request.when().delete(endpoint).then().extract().response();
    }
    attachToAllure(method, endpoint);
}

private void attachToAllure(String method, String endpoint) {
    try {
        String req = method + " " + baseUrl + endpoint;
        if (lastBody != null) req += "\n\nBody:\n" + lastBody;
        Allure.addAttachment("Request", "text/plain", req);
        Allure.addAttachment("Response [" + response.getStatusCode() + "]",
            "application/json", response.getBody().asPrettyString());
    } catch (Exception e) {
        LogUtils.debug("Allure attach falhou: " + e.getMessage());
    }
}
}

```

Decisões de design:

- O `baseUrl` é setado por instância (não estático), permitindo que diferentes Services apontem para APIs diferentes no mesmo cenário.
- O método `execute()` centraliza toda execução HTTP — logging e Allure acontecem em um único ponto. Isso evita duplicação.
- O `attachToAllure()` envolve tudo em try-catch porque o Allure não deve quebrar o teste se falhar ao anexar.
- A classe é instanciada por cenário via PicoContainer, garantindo isolamento total entre testes.
- O `lastBody` armazena o payload para exibir no attachment — útil para debug de falhas.

Dica: Nunca instancie `RestClient` manualmente. Declare-o no construtor do Step é o PicoContainer faz o resto.

3.3 Padrão Client-Service

O framework separa responsabilidades HTTP (Client) de lógica de negócio (Service). Essa separação permite que o mesmo Client seja reusado por diferentes Services, e que os Services sejam testados independentemente.

[Diagrama Mermaid — visualize no arquivo .md]

Camada

Responsabilidade

Sabe sobre HTTP?

Sabe sobre negócio?

Steps	Orquestrar acoes do cenário	Não	Indiretamente
Service	Montar requests e definir fluxos	Parcialmente	Sim
Client	Executar HTTP e capturar resposta	Sim	Não

A vantagem é clara: se amanhã a API trocar de REST para GraphQL, apenas o Client muda. Se a regra de negocio mudar (ex: campo obrigatório novo), apenas o Service muda. Os Steps permanecem intactos.

Dica: Um Service nunca deve fazer assercoes. Ele executa a acao e retorna. Quem verifica o resultado é o Step (via `restClient.getStatusCode()`).

3.4 PostService

O PostService encapsula toda a lógica de negocio relacionada ao recurso `/posts`. Ele sabe QUAIS endpoints chamar, COM QUAIS dados, mas delega a execução HTTP para o RestClient.

```
package api.services;

import api.clients.RestClient;
import utils.JsonUtils;

/**
 * Service para o recurso /posts.
 * Encapsula lógica de negocio das chamadas API.
 */
public class PostService {

    private final RestClient client;

    public PostService(RestClient client) {
        this.client = client;
    }

    public void listAll() {
        client.get("/posts");
    }

    public void getById(int id) {
        client.get("/posts/" + id);
    }

    public void getByUser(int userId) {
        client.get("/posts?userId=" + userId);
    }

    public void create() {
```

```

        String body = JsonUtils.load("payloads/posts/create-post.json");
        client.setBody(body);
        client.post("/posts");
    }

    public void update(int id) {
        String body = JsonUtils.load("payloads/posts/update-post.json")
            .replace("\"id\":1", "\"id\":" + id);
        client.setBody(body);
        client.put("/posts/" + id);
    }

    public void delete(int id) {
        client.delete("/posts/" + id);
    }
}

```

Decisões de design:

- O Service recebe o `RestClient` via construtor (injeção de dependência). Isso permite que o PicoContainer gerencie o ciclo de vida e garanta que Steps e Service compartilhem a MESMA instância do client.
- Payloads são carregados de arquivos JSON externos via `JsonUtils.load()`. Isso separa dados de lógica e facilita manutenção.
- O `update()` faz replace do ID no JSON — uma estratégia simples para parametrizar payloads sem montar objetos complexos.
- Cada método tem nome semântico: `create()`, `listAll()`, `getByUser()`. Os Steps ficam legíveis: `postService.create()`.

Dica: Para APIs com muitos endpoints (10+), considere dividir em sub-services: `PostQueryService` (GETs) e `PostCommandService` (POST/PUT/DELETE).

3.5 PostRequest — Modelo POJO

O PostRequest é um Plain Old Java Object (POJO) que representa a estrutura de dados de um post. Ele serve como modelo tipado quando você precisa construir payloads programaticamente em vez de carrega-los de arquivo.

```

package api.models;

/**
 * Modelo de request para Post.
 */
public class PostRequest {

    private String title;
    private String body;
}

```

```

    private int userId;

    public PostRequest() {}

    public PostRequest(String title, String body, int userId) {
        this.title = title;
        this.body = body;
        this.userId = userId;
    }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getBody() { return body; }
    public void setBody(String body) { this.body = body; }
    public int getUserId() { return userId; }
    public void setUserId(int userId) { this.userId = userId; }

    public String toJson() {
        return "{\"title\": \"" + title + "\", \"body\": \"" + body + "\", \"userId\": " + userId + "\"}";
    }
}

```

Decisões de design:

- O construtor vazio permite deserialização por bibliotecas como Jackson/Gson. O construtor com argumentos permite criação direta.
- O método `toJson()` gera JSON manualmente — simples e sem dependência de biblioteca de serialização. Para POJOs pequenos isso é suficiente.
- Getters e setters seguem a convenção JavaBeans, garantindo compatibilidade com qualquer framework de serialização.
- O campo `body` no POJO tem o mesmo nome do campo no JSON da API. Essa correspondência é intencional para facilitar mapeamento.

Dica: Para APIs com payloads grandes (10+ campos), considere usar Jackson com `ObjectMapper` em vez de `toJson()` manual. A chance de erro com concatenação de strings aumenta com a complexidade.

3.6 PostBuilder — Builder Pattern com Faker

O PostBuilder implementa o Builder Pattern combinado com JavaFaker para gerar dados de teste dinâmicos. Ele permite criar posts com dados aleatórios (padrão) ou customizados (quando o cenário exige valores específicos).

```

package api.builders;

import api.models.PostRequest;
import com.github.javafaker.Faker;

```

```

import java.util.Locale;

/**
 * Builder para dados de Post.
 * Gera dados dinâmicos via Faker ou permite customizacao.
 */
public class PostBuilder {

    private static final Faker faker = new Faker(new Locale("pt-BR"));

    private String title;
    private String body;
    private int userId;

    private PostBuilder() {
        this.title = faker.lorem().sentence(5);
        this.body = faker.lorem().paragraph(2);
        this.userId = 1;
    }

    public static PostBuilder valid() {
        return new PostBuilder();
    }

    public PostBuilder withTitle(String title) {
        this.title = title;
        return this;
    }

    public PostBuilder withBody(String body) {
        this.body = body;
        return this;
    }

    public PostBuilder withUserId(int userId) {
        this.userId = userId;
        return this;
    }

    public PostRequest build() {
        return new PostRequest(title, body, userId);
    }
}

```

Exemplo de uso:

```

// Post com dados totalmente aleatorios
PostRequest random = PostBuilder.valid().build();

// Post com titulo especifico, resto aleatorio
PostRequest custom = PostBuilder.valid()
    .withTitle("Meu Titulo Fixo")
    .withUserId(5)
    .build();

```

Decisoes de design:

- O construtor é privado — a única forma de criar um Builder é via `PostBuilder.valid()`. Isso garante que todo post começa com dados válidos.
- O Faker usa `Locale("pt-BR")` para gerar dados em português (nomes, textos).
- O padrão Builder (`withX().withY().build()`) é mais legível que construtores com muitos parâmetros.
- A separação Builder/Model permite ter múltiplos builders para o mesmo modelo:
`PostBuilder.valid()`, `PostBuilder.invalid()`, `PostBuilder.minimal()`.

Dica: Dados dinâmicos (Faker) evitam o problema de "dados viciados" — quando testes passam apenas com dados específicos e falham com dados reais.

3.7 JsonUtils

O `JsonUtils` é um utilitário para carregar arquivos JSON do classpath. Ele abstrai a leitura de arquivos e lança exceção semântica quando o arquivo não existe.

```
package utils;

import exceptions.FrameworkException;

import java.io.InputStream;
import java.util.Scanner;

/**
 * Utilitário para leitura de arquivos JSON do classpath.
 */
public class JsonUtils {

    private JsonUtils() {}

    /**
     * Carrega um arquivo JSON do classpath.
     * @param path caminho relativo (ex: "payloads/posts/create-post.json")
     */
    public static String load(String path) {
        InputStream input = JsonUtils.class.getClassLoader().getResourceAsStream(path);
        if (input == null) {
            throw new FrameworkException("Arquivo JSON não encontrado: " + path);
        }
        try (Scanner scanner = new Scanner(input, "UTF-8")) {
            return scanner.useDelimiter("\\A").next();
        }
    }
}
```

Decisões de design:

- Usa `getClassLoader().getResourceAsStream()` que busca no classpath (dentro de `src/test/resources/`). Isso funciona tanto local quanto em JARs.

- O `Scanner` com delimiter `\\A` (inicio do input) le o arquivo inteiro de uma vez — tecnica concisa para Java 8.
- Lanca `FrameworkException` com mensagem clara em vez de retornar null. Falhar rapido com mensagem explicativa e melhor que `NullPointerException` 5 linhas depois.
- Construtor privado porque a classe e utilitaria (apenas métodos estáticos).

Dica: Sempre use caminhos relativos ao classpath (ex: `payloads/posts/create-post.json`), nunca caminhos absolutos do sistema de arquivos. Isso garante portabilidade entre maquinas.

3.8 Payloads externalizados

Payloads JSON ficam em arquivos separados dentro de `src/test/resources/payloads/`. Isso permite que analistas editem dados sem mexer em código Java, e facilita versionamento.

create-post.json:

```
{
  "title": "Post de Teste Automatizado",
  "body": "Conteudo via REST Assured",
  "userId": 1
}
```

update-post.json:

```
{
  "id": 1,
  "title": "Titulo Atualizado",
  "body": "Corpo atualizado",
  "userId": 1
}
```

Quando usar arquivo JSON vs Builder:

O framework oferece duas estratégias para gerar payloads. A escolha depende do cenário:

- **Arquivo JSON:** Para dados fixos e determinísticos que o cenário valida explicitamente (ex: "o titulo deve ser X").
- **Builder + Faker:** Para dados dinâmicos onde o valor exato não importa (ex: teste de performance, carga).

O `PostService.create()` usa arquivo JSON porque a feature valida o titulo retornado. Se usasse Faker, o titulo seria aleatorio e a assercao `"title" deve ter valor "Post de Teste Automatizado"` falharia.

Dica: Nomeie payloads com o padrão `{acao}-{recurso}.json`. Isso cria uma convencao previsivel: `create-user.json`, `update-order.json`, `partial-update-product.json`.

3.9 JSON Schema Validation

A validação de schema garante que a API retorna a estrutura correta independentemente dos VALORES. E um teste de contrato: a API prometeu retornar campos X, Y, Z com tipos especificos.

post-schema.json:

```
{
  "type": "object",
  "required": ["userId", "id", "title", "body"],
  "properties": {
    "userId": { "type": "integer", "minimum": 1 },
    "id": { "type": "integer", "minimum": 1 },
    "title": { "type": "string", "minLength": 1 },
    "body": { "type": "string", "minLength": 1 }
  },
  "additionalProperties": true
}
```

Step de validação:

```
@E("a resposta deve estar de acordo com o schema {string}")
public void validateSchema(String schemaFile) {
    restClient.getResponse().then().assertThat()
        .body(io.restassured.module.json.JsonSchemaValidator
            .matchesJsonSchemaInClasspath("schemas/" + schemaFile));
}
```

O que o schema valida:

Regra	Significado
<code>"required": [...]</code>	Esses campos DEVEM existir na resposta
<code>"type": "integer"</code>	O campo deve ser numerico inteiro
<code>"type": "string"</code>	O campo deve ser texto
<code>"minimum": 1</code>	O valor numerico deve ser ≥ 1
<code>"minLength": 1</code>	A string não pode ser vazia

"additionalProperties": true

Permite campos extras (não quebra se API adicionar campos)

A validação de schema e complementar a validação de valores. O schema garante ESTRUTURA, os outros steps garantem CONTEUDO.

Dica: Coloque "additionalProperties": true nos schemas. Isso evita que o teste quebre quando a API adiciona um campo novo (evolucao normal de APIs).

3.10 Feature de API em portugues

A feature de API segue a mesma estrutura BDD da UI, mas sem interação visual. Cada cenário valida um endpoint específico com assercoes sobre status code, campos e estrutura.

```
# language: pt
@api
Funcionalidade: API de Posts
  Como consumidor da API REST
  Quero validar os endpoints de posts
  Para garantir que a API responde corretamente

  Contexto:
    Dado que estou consumindo a API de posts

  @smoke
  Cenário: Listar todos os posts
    Quando busco todos os posts
    Então o status code da resposta deve ser 200
    E o Content-Type da resposta deve conter "application/json"
    E a resposta deve conter 100 posts

  @smoke
  Cenário: Buscar um post por ID
    Quando busco o post de ID 1
    Então o status code da resposta deve ser 200
    E o campo "userId" deve ter valor inteiro 1
    E o campo "id" deve ter valor inteiro 1
    E o campo "title" não deve estar vazio
    E o campo "body" não deve estar vazio

  Cenário: Buscar posts de um usuário específico
    Quando busco os posts do usuário 1
    Então o status code da resposta deve ser 200
    E todos os posts devem ter "userId" igual a 1

  Cenário: Criar um novo post
    Dado que tenho os dados de um novo post
    Quando envio o novo post
    Então o status code da resposta deve ser 201
    E o campo "title" deve ter valor de texto "Post de Teste Automatizado"
    E o campo "userId" deve ter valor inteiro 1
    E o campo "id" não deve estar vazio
```

```

Cenário: Atualizar um post existente
  Dado que tenho os dados de atualização do post 1
  Quando atualizo o post 1
  Então o status code da resposta deve ser 200
  E o campo "title" deve ter valor de texto "Titulo Atualizado"

Cenário: Deletar um post
  Quando deleteo o post 1
  Então o status code da resposta deve ser 200

Cenário: Buscar post inexistente retorna 404
  Quando busco o post de ID 9999
  Então o status code da resposta deve ser 404

@smoke
Cenário: Validar contrato (schema) do post
  Quando busco o post de ID 1
  Então o status code da resposta deve ser 200
  E a resposta deve estar de acordo com o schema "post-schema.json"

```

Decisoes de design:

- O Contexto configura a API uma unica vez para todos os cenários (define baseUrl).
- Cenários cobrem o CRUD completo: Create (POST 201), Read (GET 200), Update (PUT 200), Delete (DELETE 200).
- O cenário 404 valida o caminho negativo — essencial para garantir que a API retorna erro adequado.
- Steps de validação sao genericos ("o campo X deve ter valor Y") — reusaveis para qualquer endpoint.

Dica: Organize cenários do mais simples ao mais complexo: primeiro GETs (sem body), depois POST/PUT (com body), depois DELETE, depois negativos.

3.11 PostSteps

O PostSteps é a classe mais completa de steps do framework. Ela demonstra injeção de multiplas dependências, validacoes genericas reusaveis e integração entre Service e Client.

```

package steps.api;

import api.clients.RestClient;
import api.services.PostService;
import config.Environment;
import io.cucumber.java.pt.Dado;
import io.cucumber.java.pt.E;
import io.cucumber.java.pt.Então;
import io.cucumber.java.pt.Quando;
import org.junit.Assert;

```

```
import java.util.List;

/**
 * Steps de API Posts.
 * Todas as dependências injetadas via PicoContainer.
 */
public class PostSteps {

    private final Environment env;
    private final RestClient restClient;
    private final PostService postService;

    public PostSteps(Environment env, RestClient restClient, PostService postService) {
        this.env = env;
        this.restClient = restClient;
        this.postService = postService;
    }

    @Dado("que estou consumindo a API de posts")
    public void setupApi() {
        restClient.setBaseUrl(env.apiBaseUrl);
    }

    @Dado("que tenho os dados de um novo post")
    public void prepareNewPost() {
        postService.create();
    }

    @Dado("que tenho os dados de atualização do post {int}")
    public void prepareUpdate(int id) {
        postService.update(id);
    }

    @Quando("busco todos os posts")
    public void getAll() {
        postService.listAll();
    }

    @Quando("busco o post de ID {int}")
    public void getById(int id) {
        postService.getById(id);
    }

    @Quando("busco os posts do usuário {int}")
    public void getByUser(int userId) {
        postService.getByUser(userId);
    }

    @Quando("envio o novo post")
    public void submitPost() {
        // POST executado no @Dado via postService.create()
    }

    @Quando("atualizo o post {int}")
    public void updatePost(int id) {
        // PUT executado no @Dado via postService.update()
    }

    @Quando("deleto o post {int}")
    public void deletePost(int id) {
```

```

        postService.delete(id);
    }

    @Então("o status code da resposta deve ser {int}")
    public void validateStatus(int expected) {
        Assert.assertEquals("Status incorreto", expected, restClient.getStatusCode());
    }

    @E("o Content-Type da resposta deve conter {string}")
    public void validateContentType(String expected) {
        Assert.assertTrue("Content-Type incorreto", restClient.getContentType().contains(expected));
    }

    @E("a resposta deve conter {int} posts")
    public void validateCount(int expected) {
        Assert.assertEquals("Quantidade incorreta", expected,
            restClient.getResponse().jsonPath().getList("$").size());
    }

    @E("o campo {string} deve ter valor inteiro {int}")
    public void validateIntField(String field, int expected) {
        Assert.assertEquals("Campo '" + field + "' incorreto", expected,
            restClient.getResponse().jsonPath().getInt(field));
    }

    @E("o campo {string} deve ter valor de texto {string}")
    public void validateTextField(String field, String expected) {
        Assert.assertEquals("Campo '" + field + "' incorreto", expected,
            restClient.getResponse().jsonPath().getString(field));
    }

    @E("o campo {string} não deve estar vazio")
    public void validateNotEmpty(String field) {
        Object value = restClient.getResponse().jsonPath().get(field);
        Assert.assertNotNull("Campo nulo", value);
        Assert.assertNotEquals("Campo vazio", "", value.toString().trim());
    }

    @E("todos os posts devem ter {string} igual a {int}")
    public void validateAllFields(String field, int expected) {
        List<Integer> values = restClient.getResponse().jsonPath().getList(field, Integer.class);
        Assert.assertFalse("Lista vazia", values.isEmpty());
        for (int i = 0; i < values.size(); i++) {
            Assert.assertEquals("Post[" + i + "] incorreto", expected, (int) values.get(i));
        }
    }

    @E("a resposta deve estar de acordo com o schema {string}")
    public void validateSchema(String schemaFile) {
        restClient.getResponse().then().assertThat()
            .body(io.restassured.module.jsv.JsonSchemaValidator
                .matchesJsonSchemaInClasspath("schemas/" + schemaFile));
    }
}

```

Ciclo de vida de um cenário API:

[Diagrama Mermaid — visualize no arquivo .md]

Decisoes de design:

- O construtor recebe 3 dependências (`Environment`, `RestClient`, `PostService`). O `PicoContainer` cria e injeta todas automaticamente.
- Steps de validação são genericos: "o campo X deve ter valor Y" funciona para QUALQUER endpoint que retorne JSON. Não precisamos criar steps específicos por recurso.
- O `validateAllFields` itera sobre arrays JSON — útil para validar que um filtro (`?userId=1`) realmente retornou apenas registros do usuário correto.
- Alguns `@Quando` estão vazios (ex: `submitPost()`) porque a ação já foi executada no `@Dado`. Isso é intencional para que o Gherkin leia naturalmente.

Dica: Steps genericos (validar campo, validar status) são poderosos. Crie uma classe `CommonApiSteps` com validações reusáveis e importe via `PicoContainer`.

3.12 TestData vs Payload

Existem duas estratégias para fornecer dados aos testes de API. A escolha errada causa testes frágeis ou testes que não validam nada.

Critério	Arquivo JSON (Payload)	Builder + Faker (TestData)
Dados determinísticos	Sim — valor fixo e previsível	Não — valor muda a cada execução
Cenários com asserção de valor	Ideal	Inadequado
Cenários de carga/stress	Inadequado (mesmo dado)	Ideal (dados únicos)
Cenários negativos	Bom (controle total)	Bom (customizável)
Manutenção	Baixa (editar JSON)	Média (editar Builder)
Exemplo	<code>create-post.json</code>	<code>PostBuilder.valid().build()</code>
Validação típica	"title" deve ser "X"	"title" não deve ser vazio

Regra prática:

- Se a feature Gherkin menciona um valor ESPECÍFICO na asserção → use arquivo JSON

- Se a feature valida apenas existencia/tipo → use Builder

Exemplo de cenário com Payload (valor fixo):

Então o campo "title" deve ter valor de texto "Post de Teste Automatizado"

Aqui o payload DEVE conter exatamente "Post de Teste Automatizado".

Exemplo de cenário com Builder (valor dinâmico):

Então o campo "title" não deve estar vazio

Aqui qualquer titulo serve — perfeito para Faker.

Dica: Comece com arquivos JSON (mais simples). Migre para Builder apenas quando precisar de dados dinâmicos ou multiplas variacoes do mesmo payload.

Parte 4 — Engenharia do Framework

4.1 PicoContainer — Injecao de dependência

O PicoContainer é o mecanismo de injeção de dependência (DI) do Cucumber. Ele resolve um problema fundamental: como compartilhar objetos entre diferentes classes de Steps dentro do MESMO cenário.

O que é Injecao de Dependência?

Injecao de dependência é um principio onde objetos NAO criam suas proprias dependências — eles as RECEBEM de fora. Isso é o oposto do padrão "new" direto:

```
// SEM DI – cria a propria dependência (acoplado)
public class PostSteps {
    private RestClient client = new RestClient();
}

// COM DI – recebe a dependência (desacoplado)
public class PostSteps {
    private final RestClient client;
    public PostSteps(RestClient client) {
        this.client = client;
    }
}
```

O beneficio: no segundo caso, quem controla QUAL RestClient é injetado é o container (PicoContainer). Ele pode injetar a mesma instância em multiplas classes, garantindo compartilhamento de estado.

Problema sem DI:

```
// Sem DI - cada Step cria sua propria instância
public class PostSteps {
    private RestClient client = new RestClient(); // instância A
}
public class ValidationSteps {
    private RestClient client = new RestClient(); // instância B (diferente!)
}
// PostSteps faz o request, mas ValidationSteps não ve a response
```

Solucao com PicoContainer:

```
// Com DI - mesma instância compartilhada automaticamente
public class PostSteps {
    private final RestClient client;
    public PostSteps(RestClient client) { this.client = client; } // instância A
}
public class ValidationSteps {
    private final RestClient client;
    public ValidationSteps(RestClient client) { this.client = client; } // mesma instância A!
}
```


Como funciona internamente:

1. O Cucumber detecta que `PostSteps` precisa de um `RestClient` (parametro do construtor)
2. O PicoContainer verifica se ja existe uma instância de `RestClient` neste cenário
3. Se não existe, cria uma nova (chamando o construtor padrão)
4. Se ja existe, reutiliza a mesma instância
5. Ao final do cenário, todas as instâncias sao descartadas (escopo por cenário)

Cadeia de resolução neste framework:

```
PostSteps(Environment, RestClient, PostService)
  ■■■ Environment()           → novo Environment
  ■■■ RestClient()           → novo RestClient
  ■■■ PostService(RestClient) → usa o MESMO RestClient criado acima
```

O PicoContainer e "inteligente": ele percebe que `PostService` precisa de um `RestClient`, e que um `RestClient` ja foi criado para `PostSteps`. Entao ele reutiliza a mesma instância.

Regras do PicoContainer: - Classes DEVEM ter exatamente UM construtor publico - Todas as dependências devem ser declaradas como parametros do construtor - Não precisa de anotacao, XML, ou configuração — funciona automaticamente - O escopo e POR CENARIO — a cada cenário, novas instâncias sao criadas - Tipos primitivos (int, String) não sao resolvíveis — use classes wrapper

Comparacao com outros DI containers:

Aspecto	PicoContainer	Spring	Guice
Configuração	Zero (automatico)	XML ou anotacoes	Modulos Java
Escopo	Por cenário (ideal para testes)	Configuravel	Configuravel
Overhead	Minimo	Alto	Medio
Curva de aprendizado	Minima	Alta	Media
Uso em automação	Perfeito	Excessivo	Aceitavel

Dica: Se você receber o erro "`PicoContainer cannot resolve dependency`", verifique se a classe tem um construtor sem parametros ou se todas as dependências do construtor também tem construtores resolvíveis.

4.2 ConfigReader

O ConfigReader é responsável por carregar arquivos `.properties` do classpath e resolver valores com hierarquia de prioridade (variável de ambiente > arquivo).

```
package config;

import exceptions.FrameworkException;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

/**
 * Le arquivos .properties do classpath.
 */
public class ConfigReader {

    private final Properties props = new Properties();

    public ConfigReader(String fileName) {
        try (InputStream input = getClass().getClassLoader().getResourceAsStream(fileName)) {
            if (input == null) {
                throw new FrameworkException("Arquivo não encontrado no classpath: " + fileName);
            }
            props.load(input);
        } catch (IOException e) {
            throw new FrameworkException("Erro ao carregar " + fileName, e);
        }
    }

    public String get(String key) {
        String envValue = System.getenv(key.replace(".", "_").toUpperCase());
        if (envValue != null) return envValue;
        return props.getProperty(key);
    }

    public String get(String key, String defaultValue) {
        String value = get(key);
        return value != null ? value : defaultValue;
    }

    public int getInt(String key, int defaultValue) {
        String value = get(key);
        return value != null ? Integer.parseInt(value) : defaultValue;
    }
}
```

Decisões de design:

- A hierarquia de resolução é: **variável de ambiente > arquivo .properties**. Isso permite que o CI sobreponha qualquer configuração sem alterar arquivos.
- A conversão `key.replace(".", "_").toUpperCase()` transforma `base.url` em `BASE_URL` — padrão para variáveis de ambiente.
- O `getInt()` com `defaultValue` evita `NumberFormatException` quando a propriedade não existe.

- O `FrameworkException` é lançado com mensagem clara no construtor — se o arquivo não existe, o teste falha imediatamente com diagnóstico claro.

Exemplo de override por variavel de ambiente:

```
# No CI, override a URL sem alterar dev.properties:
export BASE_URL=https://staging.example.com
```

Dica: Use variaveis de ambiente para secrets (senhas, tokens). Nunca coloque credenciais de produção em arquivos `.properties` commitados no git.

4.3 Environment

O `Environment` é a classe de fachada que determina QUAL arquivo de propriedades carregar com base no ambiente ativo. Ele implementa uma hierarquia de resolução: System Property > Variavel de ambiente > Padrão (dev).

```
package config;

/**
 * Gerencia configuracoes de ambiente.
 * Carrega o .properties correto com base em -Denvironment=dev|hml|prod
 *
 * Hierarquia:
 * 1. System Property (-Denvironment=hml)
 * 2. Variavel de ambiente (ENVIRONMENT=hml)
 * 3. Padrão: dev
 */
public class Environment {

    private final ConfigReader config;
    private final String env;

    public String baseUrl;
    public String apiBaseUrl;

    public Environment() {
        this.env = resolveEnvironment();
        this.config = new ConfigReader("environments/" + env + ".properties");
        this.baseUrl = config.get("base.url");
        this.apiBaseUrl = config.get("api.base.url");
    }

    public String get(String key) {
        return config.get(key);
    }

    public String get(String key, String defaultValue) {
        return config.get(key, defaultValue);
    }
}
```

```

    public int getInt(String key, int defaultValue) {
        return config.getInt(key, defaultValue);
    }

    public String getEnv() {
        return env;
    }

    private String resolveEnvironment() {
        if (System.getProperty("environment") != null) {
            return System.getProperty("environment");
        }
        if (System.getenv("ENVIRONMENT") != null) {
            return System.getenv("ENVIRONMENT");
        }
        return "dev";
    }
}

```

Fluxo de resolução:

```

mvn test -Denvironment=hml
→ System.getProperty("environment") = "hml"
→ Carrega: environments/hml.properties

mvn test (sem parametro)
→ System.getProperty("environment") = null
→ System.getenv("ENVIRONMENT") = null
→ Carrega: environments/dev.properties (padrão)

```

Decisões de design:

- Campos públicos `baseUrl` e `apiBaseUrl` são atalhos para as propriedades mais usadas. Evitam chamar `env.get("base.url")` dezenas de vezes.
- O `resolveEnvironment()` segue prioridade: JVM property > env var > padrão. Isso permite flexibilidade máxima.
- O padrão "dev" garante que rodar `mvn test` sem nenhum parametro funciona imediatamente (experiência zero-config para novos devs).

Dica: Em pipelines CI, use `mvn test -Denvironment=hml` para executar contra homologação. Cada ambiente pode ter URLs, timeouts e credenciais diferentes.

4.4 Configuração por ambiente

Cada ambiente tem seu próprio arquivo de propriedades. Isso permite que os mesmos testes rodem contra DEV, HML ou Produção apenas mudando um parametro.

dev.properties:

```
# =====
# Ambiente: DEV
# =====

base.url=https://opensource-demo.orangehrmlive.com/web/index.php/auth/login
api.base.url=https://jsonplaceholder.typicode.com

# Navegador
browser=chrome

# Timeouts (segundos)
timeout.implicit=10
timeout.pageLoad=30
timeout.explicit=10

# Evidências: always | failure_only
screenshot.mode=failure_only

# =====
# Credenciais de TESTE
# Em CI/CD use variaveis de ambiente (GitHub Secrets).
# Em produção use um Secrets Manager (Vault, AWS Secrets).
# =====
usuario.admin=admin
senha.admin=admin123
senha.invalida=senhaErrada
```

hml.properties:

```
# =====
# Ambiente: HML (Homologacao)
# =====

base.url=https://opensource-demo.orangehrmlive.com/web/index.php/auth/login
api.base.url=https://jsonplaceholder.typicode.com

browser=chrome
timeout.implicit=15
timeout.pageLoad=45
timeout.explicit=15
screenshot.mode=always

usuario.admin=admin
senha.admin=admin123
senha.invalida=senhaErrada
```

Diferencas entre ambientes:

Propriedade	DEV	HML
timeout.implicit	10s	15s (ambiente mais lento)
timeout.pageLoad	30s	45s

timeout.explicit	10s	15s
screenshot.mode	failure_only	always (mais evidências)

Os timeouts maiores em HML compensam a infraestrutura compartilhada (tipicamente mais lenta que DEV local). O modo de screenshot "always" gera evidências completas para auditoria.

Dica: Crie um `prod.properties` apenas com operações de leitura (GETs). Nunca execute POST/PUT/DELETE em produção automaticamente.

4.5 Logging corporativo

O framework usa SLF4J como fachada de logging e Logback como implementação. Essa separação permite trocar a implementação (ex: Log4j2) sem alterar nenhuma linha de código.

LogUtils.java:

```
package utils;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

/**
 * Logging corporativo via SLF4J + Logback.
 */
public class LogUtils {

    private static final Logger log = LoggerFactory.getLogger("automation");

    private LogUtils() {}

    public static void info(String msg) { log.info(msg); }
    public static void warn(String msg) { log.warn(msg); }
    public static void error(String msg) { log.error(msg); }
    public static void error(String msg, Throwable t) { log.error(msg, t); }
    public static void debug(String msg) { log.debug(msg); }
}
```

logback.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss} %-5level - %msg%n</pattern>
    </encoder>
  </appender>
  <appender name="FILE" class="ch.qos.logback.core.FileAppender">
```

```

<file>target/test-execution.log</file>
<encoder>
  <pattern>%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n</pattern>
</encoder>
</appender>

<logger name="automation" level="INFO"/>

<root level="WARN">
  <appender-ref ref="CONSOLE"/>
  <appender-ref ref="FILE"/>
</root>
</configuration>

```

Níveis de log e quando usar:

Nível	Quando usar	Exemplo
DEBUG	Detalhes internos (desabilitado por padrão)	"Locator encontrado: By.id('btn')"
INFO	Acoes de alto nivel	"Navegando: https://..."
WARN	Situacoes inesperadas que não impedem execução	"Timeout aguardando URL"
ERROR	Erros que comprometem o teste	"Driver não inicializou"

Decisoes de design:

- O logger usa nome fixo "automation" — um único canal para todo o framework. Isso simplifica filtragem.
- O CONSOLE usa pattern curto (HH:mm:ss) para não poluir o output. O FILE usa pattern completo com thread e data.
- O root level é WARN — bibliotecas externas (Selenium, REST Assured) só logam avisos e erros, evitando ruído.
- O arquivo vai para target/ que é limpo a cada mvn clean. Logs não acumulam indefinidamente.

Dica: Use `LogUtils.info()` em pontos estrategicos: inicio de cenário, navegação, chamadas HTTP. Isso cria um "diário de execução" que facilita debugging sem precisar rodar em modo debug.

4.6 FrameworkException

A `FrameworkException` é uma exceção customizada para erros de infraestrutura do framework. Ela sinaliza problemas que não são falhas de teste, mas sim falhas na CONFIGURACAO ou SETUP.

```
package exceptions;

/**
 * Exceção customizada do framework.
 * Usada para erros de infraestrutura (config não encontrado, template ausente, etc).
 */
public class FrameworkException extends RuntimeException {

    public FrameworkException(String message) {
        super(message);
    }

    public FrameworkException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Quando usar `FrameworkException` vs `Assert`:

Situacao	Use	Motivo
Arquivo de config não encontrado	<code>FrameworkException</code>	Problema de infra, não do teste
Status code diferente do esperado	<code>Assert.assertEquals</code>	Falha de teste (comportamento)
JSON payload ausente	<code>FrameworkException</code>	Problema de setup
Campo da resposta com valor errado	<code>Assert.assertNotNull</code>	Falha de teste
ChromeDriver não encontrado	<code>FrameworkException</code>	Problema de ambiente

Decisoes de design:

- Estende `RuntimeException` (unchecked) — não obriga try-catch em toda a cadeia. O teste simplesmente falha com stack trace claro.
- Dois construtores: um para mensagem pura, outro para "envelopar" exceções de bibliotecas (`IOException`, etc).
- A mensagem deve ser descritiva: "Arquivo não encontrado no classpath: environments/xyz.properties" e muito mais útil que "`NullPointerException`".

Dica: Crie exceções customizadas para cada tipo de falha de infra: `ConfigNotFoundException`, `DriverCreationException`. Isso facilita filtragem em reports.

4.7 Screenshots configuraveis

O sistema de screenshots é controlado por configuração de ambiente. Isso permite comportamento diferente em DEV (rapido, so falhas) e HML (completo, todas as evidências).

Implementação no UiHooks:

```
@After(value = "@ui")
public void closeBrowser(Scenário cenário) {
    WebDriver driver = DriverManager.getDriver();
    if (driver == null) return;

    String mode = env.get("screenshot.mode", "failure_only");
    boolean shouldCapture = "always".equals(mode) || cenário.isFailed();

    if (shouldCapture) {
        byte[] screenshot = ScreenshotUtils.capture(driver);
        if (screenshot.length > 0) {
            String status = cenário.isFailed() ? "FALHA" : "SUCESSO";
            cenário.attach(screenshot, "image/png", status + " - " + cenário.getName());
            LogUtils.info("Screenshot [" + status + "]");
        }
    }

    DriverManager.quit();
}
```

Fluxo de decisão:

```
screenshot.mode = "failure_only"?
    ■■■ Cenário FALHOU → Captura screenshot
    ■■■ Cenário PASSOU → Não captura (economiza tempo)

screenshot.mode = "always"?
    ■■■ Cenário FALHOU → Captura screenshot
    ■■■ Cenário PASSOU → Captura screenshot (evidência de sucesso)
```

O `cenário.attach()` do Cucumber aceita `byte[]` e automaticamente anexa a imagem ao relatório HTML e ao Allure. O nome do attachment inclui o status e o nome do cenário para facilitar navegação.

Dica: Em pipelines CI, screenshots de sucesso são úteis para validação visual por humanos. Em desenvolvimento local, elas apenas atrasam a execução.

4.8 Allure Report

O Allure gera relatórios visuais ricos com histórico de execuções, gráficos de tendência e evidências anexadas. A integração requer três componentes: dependência Maven, plugin no Runner e AspectJ.

O que o Allure oferece que o Cucumber Report não oferece:

Feature	Cucumber HTML	Allure Report
Graficos de tendencia	Não	Sim
Historico entre execucoes	Não	Sim
Categorias de falha	Não	Sim (auto-agrupamento)
Timeline de execução	Não	Sim
Attachments (screenshot, json)	Basico	Rico
Retries e flakiness	Não	Sim
Filtros e busca	Não	Sim
Ambiente/labels	Não	Sim
Severidade	Não	Sim

1. Dependência no pom.xml:

```
<dependency>
  <groupId>io.qameta.allure</groupId>
  <artifactId>allure-cucumber7-jvm</artifactId>
  <version>2.24.0</version>
</dependency>
```

2. Plugin no TestRunner:

```
plugin = {
  "io.qameta.allure.cucumber7jvm.AllureCucumber7Jvm"
}
```

3. AspectJ no Surefire:

```
<argLine>
  -javaagent:"${settings.localRepository}/org/aspectj/aspectjweaver/1.9.19/aspectjweaver-1.9.19.jar"
</argLine>
<systemPropertyVariables>
  <allure.results.directory>target/allure-results</allure.results.directory>
</systemPropertyVariables>
```

4. Plugin Maven para gerar o relatório:

```
<plugin>
  <groupId>io.qameta.allure</groupId>
  <artifactId>allure-maven</artifactId>
  <version>2.12.0</version>
</plugin>
```

Como funciona internamente:

1. Durante a execução, o plugin Cucumber gera arquivos JSON em `target/allure-results/`
2. Cada cenário gera um arquivo `-result.json` com steps, duracao, status e attachments
3. O AspectJ intercepta métodos anotados com `@Step` e registra no Allure (transparente)
4. O comando `mvn allure:serve` lê esses JSONs e gera um site HTML interativo
5. O site abre automaticamente no navegador padrão

Como visualizar o relatório:

```
mvn allure:serve
```

Este comando gera o relatório HTML e abre automaticamente no navegador. O relatório inclui: - Dashboard com taxa de sucesso/falha - Timeline mostrando duracao de cada teste - Categorias de falha agrupadas - Steps detalhados com attachments (screenshots, requests/responses) - Historico de execucoes (quando publicado via CI)

Secoas do Dashboard:

- **Overview:** Resumo geral (% passed, failed, broken, skipped)
- **Suites:** Agrupamento por feature file
- **Behaviors:** Agrupamento por funcionalidade (tag)
- **Graphs:** Graficos de status, duracao, severidade
- **Timeline:** Linha do tempo mostrando paralelismo
- **Packages:** Agrupamento por package Java

Customizacoes uteis:

Para adicionar severidade aos cenários:

```
@severity=critical
Cenário: Login com credenciais válidas
```

Para adicionar links ao Jira:

```
Allure.link("JIRA", "https://jira.empresa.com/browse/PROJ-123");
```

Para adicionar descricao:

```
Allure.description("Este cenário valida o fluxo principal de login");
```

Dica: O `mvn allure:serve` cria um servidor temporario. Para gerar arquivos estáticos permanentes, use `mvn allure:report` — o HTML fica em `target/site/allure-maven-plugin/`.

4.9 Estratégia de Tags

Tags organizam cenários em categorias executaveis independentemente. A estratégia correta de tags permite execucoes seletivas no CI e feedback rapido em desenvolvimento.

Tag	Significado	Uso
@ui	Testes de interface (Selenium)	Filtrar apenas UI
@api	Testes de API (REST Assured)	Filtrar apenas API
@smoke	Cenários criticos de regressao	Execução rapida pos-deploy
@wip	Trabalho em progresso	Excluir de CI
@bug	Bug conhecido	Documentacao

Comandos com tags:

```
# Executar apenas testes de API (rapido, sem browser)
mvn test -Dcucumber.filter.tags="@api"

# Executar apenas testes de UI
mvn test -Dcucumber.filter.tags="@ui"

# Executar smoke tests (subconjunto rapido)
mvn test -Dcucumber.filter.tags="@smoke"

# Executar smoke de API apenas
mvn test -Dcucumber.filter.tags="@api and @smoke"

# Excluir trabalho em progresso
mvn test -Dcucumber.filter.tags="not @wip"

# Combinacoes complexas
mvn test -Dcucumber.filter.tags="(@api or @ui) and @smoke and not @bug"
```

Estratégia recomendada para CI:

Gatilho	Tags executadas	Motivo
Push em feature branch	@smoke	Feedback rapido (< 2 min)
Pull Request	not @wip	Suite completa exceto WIP
Deploy em HML	@smoke com -Environment=hml	Validação pos-deploy
Noturno (scheduled)	Tudo	Regressao completa

Dica: Toda feature deve ter pelo menos 1 cenário @smoke. O smoke test é a "prova de vida" — confirma que a funcionalidade basica esta operacional.

4.10 Organização e nomenclatura

Convencoes de nomenclatura consistentes reduzem a carga cognitiva e tornam o framework previsivel. Quando qualquer QA abre o projeto, deve saber ONDE encontrar qualquer coisa.

Nomenclatura de classes:

Tipo	Padrão	Exemplo
Page Object	{Pagina}Page	LoginPage, DashboardPage
Steps	{Recurso}Steps	LoginSteps, PostSteps
Service	{Recurso}Service	PostService, UserService
Builder	{Recurso}Builder	PostBuilder, UserBuilder
Model	{Recurso}Request	PostRequest, UserRequest
Hooks	{Tipo}Hooks	UiHooks, ApiHooks
Utils	{Funcao}Utils	LogUtils, JsonUtils

Nomenclatura de métodos:

Tipo	Padrão	Exemplo
Acoes de Page	Verbo semantico	<code>fillUsername()</code> , <code>clickLogin()</code>
Consultas de Page	<code>is/get/has</code>	<code>isOnDashboard()</code> , <code>getErrorMessage()</code>
Acoes de Service	Verbo CRUD	<code>create()</code> , <code>update()</code> , <code>delete()</code>
Steps Given	Substantivo/estado	<code>openLogin()</code> , <code>setupApi()</code>
Steps When	Verbo de acao	<code>loginAsAdmin()</code> , <code>getById()</code>
Steps Then	<code>should/validate</code>	<code>shouldBeOnDashboard()</code> , <code>validateStatus()</code>

Nomenclatura de arquivos de recurso:

Tipo	Padrão	Exemplo
Feature	<code>{recurso}.feature</code>	<code>login.feature</code> , <code>posts.feature</code>
Payload	<code>{acao}-{recurso}.json</code>	<code>create-post.json</code>
Schema	<code>{recurso}-schema.json</code>	<code>post-schema.json</code>
Config	<code>{ambiente}.properties</code>	<code>dev.properties</code> , <code>hml.properties</code>

Dica: A melhor documentação e código que se auto-explica. Se você precisa de comentarios para explicar o que `mP1()` faz, renomeie para `clickMainPageButton()`.

Parte 5 — Integração e Manutenção

5.1 GitHub Actions — Pipeline completo

O pipeline CI/CD automatiza a execução dos testes a cada push ou pull request. Ele configura o ambiente (Java, Chrome, ChromeDriver), executa os testes e publica relatórios automaticamente.

```
name: Automação de Testes - Selenium + REST Assured

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

permissions:
  contents: write
  pages: write

jobs:
  testes:
    name: Executar Testes (UI + API)
    runs-on: ubuntu-latest

    steps:
      - name: Checkout do repositório
        uses: actions/checkout@v4

      - name: Configurar Java 8
        uses: actions/setup-java@v4
        with:
          java-version: '8'
          distribution: 'temurin'
          cache: maven

      - name: Instalar Google Chrome
        uses: browser-actions/setup-chrome@v1
        with:
          chrome-version: stable

      - name: Configurar ChromeDriver
        run: |
          CHROME_VERSION=$(google-chrome --version | grep -oP '\d+\.\d+\.\d+')
          echo "Chrome version: $CHROME_VERSION"
          DRIVER_URL="https://storage.googleapis.com/chrome-for-testing-public/${CHROME_VERSION}.0/"
          wget -q "$DRIVER_URL" -O /tmp/chromedriver.zip || true
          if [ -f /tmp/chromedriver.zip ]; then
            unzip -o /tmp/chromedriver.zip -d /tmp/
            sudo mv /tmp/chromedriver-linux64/chromedriver /usr/local/bin/
            sudo chmod +x /usr/local/bin/chromedriver
          fi
          chromedriver --version

      - name: Executar testes Maven
        run: mvn test --no-transfer-progress
        env:
```

```

    CI: true

  - name: Gerar relatório Allure
    uses: simple-elf/allure-report-action@master
    if: always()
    with:
      allure_results: target/allure-results
      allure_history: allure-history

  - name: Publicar Allure Report no GitHub Pages
    uses: peaceiris/actions-gh-pages@v4
    if: always()
    with:
      github_token: ${ secrets.GITHUB_TOKEN }
      publish_branch: gh-pages
      publish_dir: allure-history

  - name: Publicar relatório Cucumber
    uses: actions/upload-artifact@v4
    if: always()
    with:
      name: cucumber-report-${ github.run_number }
      path: target/cucumber-reports/
      retention-days: 30

  - name: Publicar resultado JUnit
    uses: mikepenz/action-junit-report@v4
    if: always()
    with:
      report_paths: target/cucumber-reports/cucumber.xml
      detailed_summary: true
      include_passed: true

```

Diagrama do pipeline:

[Diagrama Mermaid — visualize no arquivo .md]

Explicacao de cada step:

Step	Funcao	Por que?
Checkout	Clona o repositorio	Obter o código-fonte
Setup Java	Instala JDK 8 Temurin	Compilar e executar testes
Setup Chrome	Instala Chrome estavel	Navegador para testes UI
ChromeDriver	Instala driver compativel	Controle do Chrome via Selenium
mvn test	Executa a suite completa	CI=true ativa headless

Allure Report	Gera HTML rico	Visualizacao detalhada
GitHub Pages	Publica relatório online	Acesso via URL publica
Cucumber Artifact	Salva reports como artefato	Download por 30 dias
JUnit Report	Exibe no PR	Resultado direto no GitHub

Pontos-chave:

- `if: always( )` garante que relatórios são gerados MESMO quando testes falham. Sem isso, uma falha no `mvn test` impediria a publicação de evidências.
- `env: CI: true` faz a DriverFactory detectar ambiente CI e usar headless automaticamente.
- `cache: maven` acelera builds subsequentes reutilizando dependências baixadas.
- `workflow_dispatch` permite execução manual via interface do GitHub.

Dica: O Allure Report publicado no GitHub Pages fica acessível em `https://{usuario}.github.io/{repositorio}/`. Compartilhe essa URL com o time.

5.2 Como adicionar um novo teste UI

Siga este checklist ao adicionar um novo teste de interface ao framework:

1. **Crie o Page Object** em `pages/{funcionalidade}/{Pagina}Page.java`
2. Estenda `BasePage`
3. Declare localizadores como `private final By`

Crie métodos semânticos (não técnicos)

Crie ou edite a Feature em `resources/features/ui/{funcionalidade}.feature`

6. Use `# language: pt` na primeira linha
7. Adicione tag `@ui` na Funcionalidade

Marque cenários críticos com `@smoke`

Crie os Step Definitions em `steps/ui/{Funcionalidade}Steps.java`

10. Injete `Environment` via construtor
11. Obtenha o driver via `DriverManager.getDriver()`

Instancie a Page no primeiro @Dado

Verifique o glue no TestRunner

Confirme que `glue = {"steps", "hooks"}` cobre seu pacote

Adicione configuracoes se necessário

16. Novas URLs em `dev.properties` e `hml.properties`

Novas credenciais como propriedades

Execute localmente `bash mvn test -Dcucumber.filter.tags="@ui and @nome-da-tag"`

Verifique o relatório `bash mvn allure:serve`

Commit com mensagem descritiva `bash git add . git commit -m "feat(ui): adiciona testes de cadastro de usuario"`

Dica: Antes de commitar, execute a suite completa (`mvn test`) para garantir que o novo teste não quebrou nada existente.

5.3 Como adicionar um novo teste API

Siga este checklist ao adicionar um novo teste de API ao framework:

1. **Crie o Model (POJO)** em `api/models/{Recurso}Request.java`
2. Campos privados com getters/setters

Método `toJson()` para serializacao simples

Crie o Builder em `api/builders/{Recurso}Builder.java`

5. Construtor privado com Faker para dados padrão
6. Factory method `valid()` como ponto de entrada

Métodos `with{Campo}()` para customizacao

Crie o Service em `api/services/{Recurso}Service.java`

9. Receba `RestClient` no construtor
10. Métodos para cada operação CRUD

Carregue payloads via `JsonUtils.load()`

Crie os payloads em `resources/payloads/{recurso}/`

13. `create-{recurso}.json` para POST

`update-{recurso}.json` para PUT

Crie o schema em `resources/schemas/{recurso}-schema.json`

Defina campos required, tipos e constraints

Crie a Feature em `resources/features/api/{recurso}.feature`

18. Use `# language: pt` e tag `@api`

Cubra: GET all, GET by ID, POST, PUT, DELETE, 404

Crie os Steps em `steps/api/{Recurso}Steps.java`

21. Injete `Environment`, `RestClient`, `{Recurso}Service`

Reutilize steps genericos de validação

Adicione a apiBaseUrl (se necessário)

Se a API e diferente, adicione no `Environment`

Execute localmente `bash mvn test -Dcucumber.filter.tags="@api"`

Valide o relatório Allure

- Verifique se requests/responses estao anexados

Dica: Comece pelo cenário mais simples (GET by ID) e va adicionando complexidade. Teste cada step individualmente antes de compor o cenário completo.

5.4 Padroes de Code Review

Use esta tabela como checklist ao revisar PRs de testes automatizados:

Categoria	Verificação	Aprovado?
Nomenclatura	Classes seguem padrão (Page, Steps, Service)?	
Nomenclatura	Metodos sao semanticos (não tecnicos)?	

Page Object	Localizadores sao private final?
Page Object	Não contem assercoes (Assert)?
Steps	Usam DI via construtor (não new)?
Steps	Sao coesos (um assunto por classe)?
Feature	Tem tag @ui ou @api?
Feature	Pelo menos 1 cenário @smoke?
Feature	Cenários sao independentes entre si?
Service	Não faz assercoes?
Service	Delega HTTP para RestClient?
Config	Nenhum valor hardcoded?
Config	Credenciais vem de properties/env vars?
Geral	Nenhum <code>Thread.sleep()</code> explicito?
Geral	Nenhum <code>System.out.println()</code> ? (use LogUtils)
Geral	Testes passam localmente?
Geral	Não quebra testes existentes?

Dica: Um PR de teste deve conter: feature + steps + page/service. Se faltar alguma peça, o teste esta incompleto.

5.5 Checklist para iniciar novo projeto

Ao iniciar um novo projeto de automação baseado neste framework:

Fase 1 — Setup inicial: - ☐ Clonar o repositorio template - ☐ Atualizar `groupId` e `artifactId` no `pom.xml` - ☐ Atualizar `<name>` e `<description>` no `pom.xml` - ☐ Configurar `base.url` e `api.base.url` em `dev.properties` - ☐ Configurar credenciais (`dev.properties` ou `env vars`) - ☐ Atualizar o path do `ChromeDriver` (ou configurar `env var`) - ☐ Executar `mvn clean compile` para validar dependências - ☐

Executar `mvn test -Dcucumber.filter.tags="@api"` para validar setup básico

Fase 2 — Primeiro teste: - [] Identificar a tela/endpoint mais simples da aplicação - [] Criar a primeira Page Object da tela principal - [] Criar o primeiro cenário @smoke (o mais simples possível) - [] Executar `mvn test` e validar que passa - [] Verificar que o Allure gera relatório (`mvn allure:serve`)

Fase 3 — CI/CD: - [] Configurar GitHub Actions (copiar testes.yml) - [] Ajustar steps de CI se necessário (versão Chrome, env vars) - [] Habilitar GitHub Pages para o branch gh-pages - [] Configurar secrets no repositório (credenciais) - [] Fazer primeiro push e validar pipeline - [] Verificar que o Allure Report está acessível online

Fase 4 — Time: - [] Compartilhar URL do Allure Report com o time - [] Documentar convenções específicas do projeto - [] Criar template de PR para testes - [] Definir responsáveis por manutenção da suite - [] Agendar revisão semanal de testes flakey

Dica: O primeiro teste deve ser o mais simples possível (ex: abrir a página e verificar o título). Isso valida toda a infra antes de adicionar complexidade.

5.6 Troubleshooting

Os 10 erros mais comuns ao trabalhar com o framework e suas soluções:

#	Erro	Causa	Solução
1	<code>SessionNotCreatedException</code> <code>ChromeDriver</code> <code>version mismatch</code>	<code>ChromeDriver</code> incompatível com Chrome instalado	Baixe a versão correta em chromedriver.chromium.org
2	<code>NoSuchElementException</code>	Elemento não encontrado na página	Verifique o localizador; adicione wait explícito
3	<code>TimeoutException</code>	Elemento não apareceu no tempo configurado	Aumente <code>timeout.explicit</code> ou verifique se a página carregou
4	<code>FrameworkException:</code> Arquivo não encontrado no classpath	Arquivo <code>.properties</code> ou <code>.json</code> ausente	Verifique o caminho e se está em <code>src/test/resources/</code>

5	<code>PicoContainer: cannot instantiate</code>	Classe sem construtor resolvel	Adicione construtor publico com dependências resolvíveis
6	<code>StaleElementReferenceException</code>	Elemento ficou obsoleto (pagina recarregou)	Re-localize o elemento após navegação
7	<code>Connection refused em REST Assured</code>	API não acessível ou URL errada	Verifique <code>api.base.url</code> no <code>.properties</code>
8	<code>WebDriverException: unknown error: Chrome failed to start</code>	Chrome não instalado ou headless ausente em CI	Verifique flag <code>--headless=new</code> e <code>--no-sandbox</code>
9	<code>cucumber.runtime.CucumberException: Glue not found</code>	Package não está no glue	Adicione o package em <code>@CucumberOptions(glue = {...})</code>
10	<code>allure-results directory empty</code>	AspectJ não carregado	Verifique <code>-javaagent</code> no <code>argLine</code> do Surefire

Detalhamento dos 5 erros mais comuns:

Erro #1 — ChromeDriver version mismatch:

```
org.openqa.selenium.SessionNotCreatedException:
session not created: This version of ChromeDriver only supports Chrome version 120
Current browser version is 125.0.6422.60
```

O Chrome atualiza automaticamente, mas o ChromeDriver não. Sempre que o Chrome atualizar, você precisa baixar o ChromeDriver correspondente. Em CI, isso é automático pelo step de setup.

Erro #2 — NoSuchElementException:

```
org.openqa.selenium.NoSuchElementException:
no such element: Unable to locate element: {"method":"css selector","selector":".btn-primary"}
```

Possíveis causas: - O localizador está errado (typo no CSS selector) - O elemento ainda não carregou (falta wait) - O elemento está dentro de um iframe (precisa switchTo) - O elemento está em outra aba (precisa switchTo window)

Erro #5 — PicoContainer:

```
io.cucumber.core.exception.CucumberException:  
Failed to instantiate class steps.api.PostSteps
```

Possíveis causas: - A classe tem 2 construtores (PicoContainer exige apenas 1) - Uma dependência do construtor não tem construtor padrão - Dependência circular (A depende de B que depende de A)

Erro #6 — StaleElementReferenceException:

```
org.openqa.selenium.StaleElementReferenceException:  
stale element reference: element is not attached to the page document
```

Isso acontece quando você guarda uma referencia a um elemento (`WebElement`) e a pagina recarrega ou o DOM muda. Solução: sempre re-localize o elemento antes de interagir. No nosso framework, os métodos da `BasePage` já fazem isso (usam `By` em vez de `WebElement` armazenado).

Erro #8 — Chrome failed to start:

```
org.openqa.selenium.WebDriverException:  
unknown error: Chrome failed to start: crashed
```

Em CI Linux, isso ocorre quando faltam flags obrigatorias: - `--headless=new` — sem interface grafica - `--no-sandbox` — containers não suportam sandbox - `--disable-dev-shm-usage` — previne crash por memoria compartilhada

Processo de investigacao:

1. Leia a mensagem de erro COMPLETA (não apenas a primeira linha)
2. Identifique se e erro de INFRA (`FrameworkException`) ou de TESTE (`AssertionError`)
3. Verifique os logs em `target/test-execution.log`
4. Se for UI, verifique a screenshot anexada ao relatório
5. Se for API, verifique o request/response no Allure

Dica: 80% dos problemas de CI são de compatibilidade Chrome/ChromeDriver. Quando o pipeline quebra sem motivo aparente, verifique se o Chrome atualizou automaticamente.

5.7 FAQ

1. Posso rodar testes em paralelo? Sim. O `DriverManager` com `ThreadLocal` já suporta paralelismo. Configure o Surefire com `<parallel>methods</parallel>` ou use o Cucumber paralelo nativo.

2. Como adicionar outro navegador (Firefox)? Adicione um case `"firefox"` no switch da `DriverFactory.create()` com `FirefoxDriver` e `FirefoxOptions`. Altere `browser=firefox` no

.properties.

3. Como rodar contra uma API autenticada? Use `restClient.addHeader("Authorization", "Bearer " + token)` no step de setup. O token pode vir do `Environment` ou de um step anterior de login.

4. Como ignorar um teste temporariamente? Adicione a tag `@wip` no cenário e execute com `-Dcucumber.filter.tags="not @wip"`.

5. Como rodar apenas 1 cenário específico? Crie uma tag unica (`@focus`) no cenário e execute com `-Dcucumber.filter.tags="@focus"`. Remova antes de commitar.

6. Por que usar JUnit 4 e não JUnit 5? O Cucumber 7 suporta ambos, mas JUnit 4 com `@RunWith(Cucumber.class)` e mais simples de configurar e compatível com todas as versoes do Surefire.

7. Como debugar um teste que passa local e falha no CI? Verifique: (a) timeouts — CI e mais lento, (b) resolução de tela — headless pode ter layout diferente, (c) dados — APIs podem retornar dados diferentes.

8. Posso usar Selenium 4? Selenium 4 requer Java 11+. Se migrar para Java 11, atualize `selenium.version` e remova a configuração manual do `ChromeDriver` (Selenium 4 tem `SeleniumManager` built-in).

9. Como gerar dados únicos por execução? Use o `PostBuilder` com `JavaFaker`. Cada chamada a `PostBuilder.valid().build()` gera titulo e corpo diferentes.

10. Como reportar um bug encontrado pelo teste? Marque o cenário com `@bug(JIRA-123)` para documentar. O relatório Allure agrupa por tags, facilitando rastreabilidade.

Dica: Mantenha este FAQ atualizado com perguntas reais do time. E a documentação mais consultada de qualquer framework.

5.8 Glossario

Termo	Definicao
BDD	Behavior-Driven Development — metodologia que descreve comportamento em linguagem natural
Cucumber	Framework que executa especificacoes BDD escritas em Gherkin
Gherkin	Linguagem de especificação com keywords (Dado/Quando/Entao)

Page Object	Padrão que encapsula elementos e ações de uma página em uma classe
Step Definition	Método Java que implementa uma frase Gherkin
Hook	Código executado antes (@Before) ou depois (@After) de cenários
Feature	Arquivo .feature que descreve uma funcionalidade em Gherkin
Cenário	Caso de teste individual dentro de uma Feature
WebDriver	Interface do Selenium para controlar navegadores
REST Assured	Biblioteca Java para testes de API REST
ThreadLocal	Mecanismo Java que isola variáveis por thread
PicoContainer	Framework leve de injeção de dependência
Allure	Framework de reporting com interface visual rica
Headless	Modo de execução de navegador sem interface gráfica
CI/CD	Continuous Integration / Continuous Delivery — automação de build e deploy
Pipeline	Sequência automatizada de steps (build, test, deploy)
Smoke Test	Subconjunto mínimo de testes que valida funcionalidade básica
Schema Validation	Validação da ESTRUTURA (não valores) de uma resposta JSON
Builder Pattern	Padrão de design para construção fluente de objetos complexos
POJO	Plain Old Java Object — classe simples sem dependências de framework

5.9 Comandos rapidos

Execução de testes:

Comando	Descricao
<code>mvn test</code>	Executa todos os testes
<code>mvn test -Dcucumber.filter.tags="@smoke"</code>	Apenas smoke tests
<code>mvn test -Dcucumber.filter.tags="@api"</code>	Apenas testes de API
<code>mvn test -Dcucumber.filter.tags="@ui"</code>	Apenas testes de UI
<code>mvn test -Dcucumber.filter.tags="@api and @smoke"</code>	Smoke tests de API
<code>mvn test -Dcucumber.filter.tags="not @wip"</code>	Tudo exceto WIP
<code>mvn test -Denvironment=hml</code>	Executar contra HML
<code>mvn test -Denvironment=hml -Dcucumber.filter.tags="@smoke"</code>	Smoke em HML

Relatórios:

Comando	Descricao
<code>mvn allure:serve</code>	Gera e abre Allure no navegador
<code>mvn allure:report</code>	Gera HTML estático em target/site/

Utilitarios Maven:

Comando	Descricao
<code>mvn clean</code>	Limpa pasta target/
<code>mvn clean test</code>	Limpa e executa tudo
<code>mvn dependency:tree</code>	Mostra arvore de dependências

<code>mvn versions:display-dependency-updates</code>	Mostra atualizações disponíveis
<code>mvn compile -X</code>	Compilação com debug verbose

Variáveis de ambiente úteis:

Variável	Valor	Efeito
<code>CI=true</code>	Qualquer valor	Ativa modo headless
<code>ENVIRONMENT=hml</code>	dev, hml, prod	Seleciona ambiente
<code>CHROME_DRIVER_PATH=/usr/local/bin/chromedriver</code>	Caminho	Override do path do driver
<code>BASE_URL=https://staging.app.com</code>	URL	Override da URL base
<code>API_BASE_URL=https://api-staging.app.com</code>	URL	Override da URL da API

Dica: Crie aliases no seu terminal para os comandos mais usados: `alias smoke="mvn test -Dcucumber.filter.tags='@smoke'".`

5.10 Como evoluir a arquitetura

O framework atual é uma base sólida. Aqui estão os próximos passos para evolução conforme a maturidade do time e do projeto cresce:

Curto prazo (1-2 sprints): - Adicionar mais Page Objects para telas críticas - Expandir cenários de API para outros endpoints - Configurar execução noturna (cron) no GitHub Actions - Adicionar mais cenários negativos e edge cases - Criar `prod.properties` com testes read-only - Implementar cenários de login com múltiplos perfis (admin, user, guest) - Adicionar validação de headers customizados

Medio prazo (1-3 meses): - Implementar execução paralela com `maven-surefire-plugin` parallel - Adicionar suporte a Firefox e Edge na DriverFactory - Criar camada de dados dinâmicos com JavaFaker para UI - Implementar retry automático para testes flakey - Adicionar testes de acessibilidade (axe-core via Selenium) - Separar Runners: `UiRunner.java` e `ApiRunner.java` - Implementar Page Factory com `@FindBy` (alternativa ao By manual) - Criar interceptors para logging automático de requests - Adicionar health-check step antes da suite (verificar se APIs estão no ar)

Longo prazo (3-6 meses): - Migrar para Java 11+ e Selenium 4 (SeleniumManager elimina ChromeDriver manual) - Implementar testes visuais com comparação de screenshots - Adicionar testes de performance com Gatling ou k6 - Criar dashboard de métricas (taxa de flakiness, tempo médio) - Implementar BDD reverso: gerar steps a partir de Swagger/OpenAPI - Considerar migrar UI para Playwright (mais estável que

Selenium) - Implementar banco de dados de resultados para historico - Criar API interna para disparar suites sob demanda - Adicionar testes de contrato com Pact (consumer-driven contracts)

Modelo de maturidade de automação:

Nível	Descrição	Indicadores
1 - Inicial	Scripts soltos, sem framework	Poucos testes, ninguém mantém
2 - Estruturado	Framework organizado (VOCE ESTA AQUI)	Page Objects, CI, relatórios
3 - Otimizado	Paralelo, multi-browser, dados dinâmicos	<5min smoke, <15min full
4 - Integrado	Conectado com Jira, dashboards, alertas	Métricas de qualidade automáticas
5 - Preditivo	ML para detectar flakiness, auto-heal	Self-healing locators, analytics

Princípios para evolucao: - Nunca quebre a compatibilidade com testes existentes - Adicione features de forma incremental, não em big-bang - Cada evolucao deve ter sua propria branch e PR - Valide com o time antes de adotar novas bibliotecas - Documente decisões arquiteturais em ADRs (Architecture Decision Records) - Meça o impacto: tempo de execução antes e depois da mudanca - Mantenha um CHANGELOG do framework

Dica: A melhor evolucao é a que resolve uma dor REAL do time. Não adicione complexidade "porque e legal" — adicione porque resolve um problema concreto.

Apendice — Diagramas Complementares de Arquitetura

Diagrama de Dependências entre Classes

Este diagrama mostra como as classes se relacionam no framework. Setas indicam "depende de":

[Diagrama Mermaid — visualize no arquivo .md]

Mapa Mental do Framework

```
Framework de Automação
  ■■■ UI (Selenium)
  ■   ■■■ Pages (encapsulam elementos)
  ■   ■   ■■■ BasePage (métodos comuns)
  ■   ■   ■■■ LoginPage (pagina especifica)
  ■   ■■■ Drivers (gerenciam navegador)
  ■   ■   ■■■ DriverFactory (cria)
  ■   ■   ■■■ DriverManager (armazena)
  ■   ■■■ Hooks (ciclo de vida)
  ■   ■■■ UiHooks (before/after)
  ■
  ■■■ API (REST Assured)
  ■   ■■■ Clients (execução HTTP)
  ■   ■   ■■■ RestClient (corporativo)
  ■   ■■■ Services (lógica de negocio)
  ■   ■   ■■■ PostService
  ■   ■■■ Models (POJOs)
  ■   ■   ■■■ PostRequest
  ■   ■■■ Builders (dados dinâmicos)
  ■   ■■■ PostBuilder
  ■
  ■■■ Config (gerenciamento)
  ■   ■■■ ConfigReader (leitura)
  ■   ■■■ Environment (resolução)
  ■   ■■■ *.properties (arquivos)
  ■
  ■■■ Steps (Gherkin → Java)
  ■   ■■■ LoginSteps (UI)
  ■   ■■■ PostSteps (API)
  ■
  ■■■ Utils (transversais)
  ■   ■■■ LogUtils (logging)
  ■   ■■■ ScreenshotUtils (evidências)
  ■   ■■■ JsonUtils (payloads)
  ■
  ■■■ Reports (saida)
  ■   ■■■ Allure (visual)
  ■   ■■■ Cucumber HTML
  ■   ■■■ JUnit XML
```

Apendice A — Anatomia Detalhada dos Hooks

Hooks de API

O framework possui hooks específicos para cenários de API. Eles são mais simples que os de UI (não há browser para gerenciar), mas cumprem papel importante de observabilidade.

```
package hooks;

import io.cucumber.java.Before;
import io.cucumber.java.Scenário;
import utils.LogUtils;

/**
 * Hooks para cenários @api.
 */
public class ApiHooks {

    @Before(value = "@api", order = 0)
    public void logScenário(Scenário cenário) {
        LogUtils.info("=== [API] " + cenário.getName() + " ===");
    }
}
```

Por que hooks de API são minimalistas:

- Não há browser para abrir/fechar
- O RestClient é gerenciado pelo PicoContainer (criado/destruído por cenário automaticamente)
- Não há screenshots para capturar
- O Allure já captura request/response via `attachToAllure()` no RestClient

O único hook necessário é o log de início para rastreabilidade nos logs.

Comparação de complexidade:

Aspecto	UiHooks	ApiHooks
@Before	2 métodos (log + browser)	1 método (log)
@After	1 método (screenshot + quit)	Nenhum
Dependências	Environment, DriverFactory, DriverManager	Nenhuma
Linhas de código	~40	~10

Dica: Se amanhã você precisar de setup de API mais complexo (ex: criar token de auth), adicione um `@Before` no `ApiHooks`. Não polua os `Steps` com lógica de infra.

Ordem de execução dos Hooks

Quando um cenário tem multiplas tags (ex: `@ui @smoke`), os hooks executam na ordem definida pelo atributo `order`. Hooks com `order` menor executam primeiro.

```
@Before(order = 0) → Primeiro
@Before(order = 1) → Segundo
@Before(order = 2) → Terceiro
... cenário executa ...
@After(order = 2) → Primeiro (ordem INVERSA no After!)
@After(order = 1) → Segundo
@After(order = 0) → Ultimo
```

Atencao: No `@After`, a ordem e INVERSA — hooks com `order` MAIOR executam primeiro. Isso segue a lógica de "pilha": o ultimo a abrir é o primeiro a fechar.

Apêndice B — Padrões de Design Aplicados

Este framework aplica diversos padrões de design (Design Patterns) do catálogo GoF e padrões específicos de automação. Entender esses padrões permite que você os aplique em outros contextos.

Page Object Pattern

Problema: Testes UI tem localizadores (By.id, By.css) espalhados por todo o código. Quando a UI muda, dezenas de testes quebram.

Solução: Encapsular todos os localizadores e interações de uma página em uma única classe. Quando a UI muda, apenas a Page precisa de atualização.

Aplicação no framework:

```
LoginPage.java
■■■ usernameField (By)      → Localizador encapsulado
■■■ passwordField (By)     → Localizador encapsulado
■■■ fillUsername(String)   → Interação semântica
■■■ clickLogin()           → Interação semântica
■■■ isOnDashboard()        → Consulta de estado
```

Benefício quantificado: Se o seletor do botão de login mudar, você altera 1 linha em `LoginPage.java`. Sem Page Object, alteraria em TODOS os testes que fazem login.

Factory Pattern

Problema: A criação de WebDriver envolve lógica complexa (opções, headless, CI detection). Essa lógica não deve ficar nos Hooks.

Solução: A `DriverFactory` encapsula toda a lógica de criação. O Hook apenas chama `factory.create("chrome")`.

Aplicação no framework:

```
DriverFactory.create("chrome")
■■■ Detecta CI? → headless + no-sandbox
■■■ Detecta local? → maximized + path do driver
■■■ Retorna WebDriver pronto
```

Builder Pattern

Problema: Criar objetos com muitos campos (PostRequest) via construtor é confuso: `new PostRequest("titulo", "corpo", 1)` — qual parâmetro é qual?

Solucao: O `PostBuilder` usa interface fluente para construção legível.

Aplicacao no framework:

```
PostBuilder.valid()           // Dados aleatorios padrão
    .withTitle("Meu Post") // Override específico
    .withUserId(5)          // Override específico
    .build();               // Retorna PostRequest
```

Singleton (ThreadLocal)

Problema: O `WebDriver` precisa ser acessível por Hooks, Steps e Pages sem ser passado como parametro em toda a cadeia.

Solucao: `DriverManager` usa `ThreadLocal` — um "singleton por thread" que permite acesso global seguro.

Template Method

Problema: Todas as Pages precisam de métodos comuns (type, click, getText) mas com implementacoes de localizadores diferentes.

Solucao: `BasePage` define o template (métodos protegidos) e cada Page filha usa esses métodos com seus proprios localizadores.

Apendice C — Exemplos de Cenários Avancados

Cenário com Data Table

O Cucumber suporta tabelas de dados para passar múltiplos valores a um step:

```
Cenário: Criar post com dados customizados
Dado que tenho os seguintes dados para o post:
  | campo | valor |
  | title | Post via Data Table |
  | body  | Corpo do post |
  | userId | 1 |
Quando envio o novo post
Então o status code da resposta deve ser 201
```

Step correspondente:

```
@Dado("que tenho os seguintes dados para o post:")
public void preparePostFromTable(io.cucumber.datatable.DataTable dataTable) {
    Map<String, String> data = dataTable.asMap(String.class, String.class);
    PostRequest post = new PostRequest(
        data.get("title"),
        data.get("body"),
        Integer.parseInt(data.get("userId"))
    );
    restClient.setBody(post.toJson());
}
```

Cenário com Cenário Outline e múltiplas variações

```
Esquema do Cenário: Validar status codes por endpoint
Quando faço GET no endpoint "<endpoint>"
Então o status code da resposta deve ser <status>
```

Exemplos:

endpoint	status
/posts	200
/posts/1	200
/posts/9999	404
/invalid	404

Isso gera 4 testes independentes a partir de um único template, cada um com seus próprios dados e resultado esperado.

Cenário com Hooks condicionais

```
@ui @auth-required
Cenário: Acessar pagina protegida
  Dado que estou autenticado como administrador
  Quando acesso a pagina de configuracoes
  Então devo ver o painel de administracao
```

Com um hook condicional:

```
@Before(value = "@auth-required", order = 2)
public void authenticate() {
    // Login automatico antes de cenários que precisam de auth
}
```

Apendice D — Metricas e Indicadores

KPIs de Automação

Acompanhe estas metricas para medir a saude do framework:

Metrica	Meta	Como medir
Taxa de sucesso	> 95%	Cenários passando / total
Tempo de execução (smoke)	< 3 min	Duracao do job @smoke
Tempo de execução (full)	< 15 min	Duracao do job completo
Taxa de flakiness	< 5%	Testes instáveis / total
Cobertura de funcionalidades	> 80%	Features automatizadas / total
MTTR (tempo para corrigir)	< 1 dia	Tempo entre falha e fix
Taxa de falsos positivos	< 2%	Falhas de infra / total de falhas

Como calcular ROI da automação

```
ROI = (Custo Manual - Custo Automatizado) / Custo Automatizado x 100
```

```
Custo Manual = Execucoes/sprint x Tempo/execução x Custo/hora
```

```
Custo Automatizado = Tempo de criação + Manutenção mensal
```

Exemplo:

- 50 execucoes/sprint x 30 min x R\$50/h = R\$1.250/sprint (manual)
- Criação: 40h x R\$80/h = R\$3.200 (investimento único)
- Manutenção: 4h/sprint x R\$80/h = R\$320/sprint
- Payback: 3.200 / (1.250 - 320) = 3.4 sprints

Apos ~4 sprints, a automação se paga é a partir dai gera economia pura.

Dica: Apresente o ROI para a gestao usando numeros reais do seu time. E a forma mais eficaz de justificar investimento em automação.

Apêndice E — Padrões de Localizadores

Hierarquia de preferência

Nem todos os localizadores são iguais. Use a hierarquia abaixo para escolher o mais resistente a mudanças:

Prioridade	Tipo	Exemplo	Estabilidade
1 (melhor)	ID semântico	<code>By.id("login-btn")</code>	Alta
2	Name	<code>By.name("username")</code>	Alta
3	data-testid	<code>By.cssSelector("[data-testid='submit']")</code>	Média
4	CSS seletor curto	<code>By.cssSelector("button[type='submit']")</code>	Média
5	CSS seletor longo	<code>By.cssSelector("div.form > button.primary")</code>	Baixa
6 (pior)	XPath absoluto	<code>By.xpath("/html/body/div[1]/div/form/button")</code>	Muito Baixa

Regras práticas:

- Se o elemento tem `id` ou `name`, USE. São os mais estáveis.
- Se não tem, peça ao dev para adicionar `data-testid` — é um contrato entre QA e DEV.
- Evite XPath absoluto a todo custo — qualquer mudança no DOM quebra.
- CSS selectors com atributos (`[type='submit']`) são mais estáveis que selectors estruturais (`.parent > .child`).

Exemplo real do framework:

```
// BOM - usa atributos semanticos
private final By usernameField = By.name("username");
private final By loginButton = By.cssSelector("button[type='submit']");

// RUIM - depende da estrutura do DOM
private final By loginButton = By.xpath("//div[@class='login-form']/div[3]/button");
```

Dica: Crie uma convenção com o time de desenvolvimento: todo elemento interativo DEVE ter um `data-testid`. Isso elimina 90% dos problemas de localizadores.

Apendice F — Boas Praticas de Gherkin

O que fazer é o que evitar

Fazer	Evitar
Cenários independentes	Cenários que dependem de ordem
Steps declarativos ("facio login")	Steps imperativos ("clico no campo, digito X")
1 assercao principal por cenário	10 assercoes em 1 cenário
Nomes descritivos no cenário	"Teste 1", "Cenário feliz"
Background para setup comum	Repetir Dado em todo cenário
Tags para organização	Features sem tags

Anti-padroes comuns

Anti-padrão 1: Steps muito tecnicos

```
# RUIM - linguagem tecnica
Dado que navego para "https://app.com/login"
Quando digito "admin" no campo "#username"
E clico no elemento "button.submit"

# BOM - linguagem de negocio
Dado que estou na pagina de login
Quando facio login como administrador
```

Anti-padrão 2: Cenários dependentes

```
# RUIM - cenário 2 depende do cenário 1 ter executado
Cenário: Criar usuario
...
Cenário: Editar usuario criado acima
...

# BOM - cada cenário e independente
Cenário: Editar usuario existente
  Dado que existe um usuario com ID 1
  ...
```

Anti-padrão 3: Feature gigante

```
# RUIM - 30 cenários em 1 feature
Funcionalidade: Sistema completo
  Cenário: Login ...
  Cenário: Cadastro ...
  Cenário: Pagamento ...

# BOM - features focadas
# login.feature, cadastro.feature, pagamento.feature
```

Dica: Cada feature deve ter entre 3 e 10 cenários. Se passar de 10, provavelmente pode ser dividida em features menores e mais coesas.

Consideracoes Finais

Este framework representa uma base profissional para automação de testes que equilibra simplicidade de uso com robustez arquitetural. Cada decisão — desde a escolha do Java 8 ate a estratégia de screenshots configuraveis — foi tomada pensando em cenários reais de empresas brasileiras.

O mais importante não e decorar cada classe, mas entender os PRINCIPIOS por tras delas:

1. **Separação de responsabilidades** — cada classe faz UMA coisa bem
2. **Configuração externalizada** — nenhum valor fixo no código
3. **Injecao de dependência** — objetos sao recebidos, não criados
4. **Reusabilidade** — steps genericos, Pages herdam de base
5. **Observabilidade** — logs, screenshots e reports em toda execução

Com esses principios internalizados, você consegue construir e MANTER frameworks profissionais em qualquer tecnologia.

Apendice G — REST Assured em Profundidade

Metodos HTTP e seus significados

Metodo	Semantica	Idempotente?	Corpo na request?	Uso tipico
GET	Ler recurso	Sim	Não	Buscar dados
POST	Criar recurso	Não	Sim	Inserir novo registro
PUT	Substituir recurso	Sim	Sim	Atualizar completamente
PATCH	Atualizar parcialmente	Não	Sim	Alterar campos especificos
DELETE	Remover recurso	Sim	Opcional	Excluir registro

Idempotente significa que executar a mesma operação N vezes produz o mesmo resultado. GET e PUT sao seguros para retry automatico; POST não e (pode criar duplicatas).

Validacoes avancadas com JsonPath

O REST Assured permite extrair e validar qualquer parte de uma resposta JSON usando JsonPath:

```
// Validar campo simples
response.jsonPath().getString("title")           // "Post de Teste"
response.jsonPath().getInt("userId")              // 1

// Validar array
response.jsonPath().getList("$")                  // Lista de objetos
response.jsonPath().getList("$").size()           // Tamanho do array

// Validar campo em posicao especifica do array
response.jsonPath().getString("[0].title")        // Titulo do primeiro item

// Filtrar array
response.jsonPath().getList("findAll { it.userId == 1 }") // Posts do user 1

// Validar campo aninhado
response.jsonPath().getString("address.city")     // Campo dentro de objeto
```

Status Codes que você precisa conhecer

Código	Significado	Quando validar
200	OK (sucesso)	GET, PUT, PATCH, DELETE
201	Created	POST (recurso criado)
204	No Content	DELETE (sem corpo na resposta)
400	Bad Request	Payload invalido
401	Unauthorized	Token ausente/invalido
403	Forbidden	Sem permissao
404	Not Found	Recurso inexistente
405	Method Not Allowed	Verbo HTTP não suportado
409	Conflict	Duplicidade (ex: email ja existe)
422	Unprocessable Entity	Validação de negocio falhou
500	Internal Server Error	Bug no servidor

Dica: Sempre teste cenários negativos (400, 401, 404). Eles são tão importantes quanto os positivos — garantem que a API rejeita inputs inválidos corretamente.

Apendice H — Estratégias de Espera (Waits)

Os tres tipos de espera no Selenium

O gerenciamento de esperas é a causa #1 de testes flakey (instáveis). Entender os tres tipos e fundamental.

Tipo	Como funciona	Vantagem	Desvantagem
Implicit Wait	Espera X segundos em TODA busca de elemento	Simples de configurar	Espera mesmo quando não precisa
Explicit Wait	Espera ATE uma condicao ser verdadeira	Preciso e inteligente	Requer WebDriverWait
Thread.sleep()	Para a execução por tempo fixo	Nenhuma	NUNCA use em testes

Implicit Wait (configurado no UiHooks):

```
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
```

Toda vez que `driver.findElement()` é chamado, o Selenium espera ate 10 segundos pelo elemento aparecer. Se aparecer em 1 segundo, continua imediatamente.

Explicit Wait (usado na BasePage):

```
WebDriverWait wait = new WebDriverWait(driver, 10);  
wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
```

Espera ate 10 segundos ESPECIFICAMENTE por aquele elemento ficar visivel. Mais preciso que implicit.

Por que NUNCA usar Thread.sleep():

```
// RUIM - espera 5 segundos SEMPRE, mesmo se o elemento ja apareceu  
Thread.sleep(5000);  
driver.findElement(By.id("btn")).click();  
  
// BOM - espera NO MAXIMO 10 segundos, continua assim que aparece  
wait.until(ExpectedConditions.elementToBeClickable(By.id("btn"))).click();
```

O `Thread.sleep()` desperdiça tempo (elemento aparece em 0.5s, mas espera 5s) e ao mesmo tempo não garante sucesso (se o elemento levar 6s, falha).

ExpectedConditions mais usadas

Condicao	Quando usar
<code>visibilityOfElementLocated(locator)</code>	Quando precisa ver/ler o elemento
<code>elementToBeClickable(locator)</code>	Antes de clicar
<code>invisibilityOfElementLocated(locator)</code>	Esperar loading sumir
<code>urlContains(fragment)</code>	Aguardar navegação
<code>textToBePresentInElement(elem, text)</code>	Aguardar texto dinâmico
<code>alertIsPresent()</code>	Aguardar popup JavaScript
<code>frameToBeAvailableAndSwitchToIt(frame)</code>	Aguardar iframe carregar

Dica: No nosso framework, você nunca chama *ExpectedConditions* diretamente nos Steps. Os métodos da *BasePage* (*type*, *click*, *getText*) já encapsulam as esperas adequadas.

Apendice I — Gerenciamento de Dados de Teste

Estratégias de dados

Estratégia	Descrição	Quando usar
Dados fixos em .properties	Credenciais, URLs	Configuração de ambiente
Dados fixos em JSON	Payloads determinísticos	Cenários com assertão de valor
Dados dinâmicos (Faker)	Valores aleatórios por execução	Cenários de existência
Dados parametrizados	Tabela de Exemplos no Gherkin	Variações do mesmo fluxo
Dados compartilhados (Context)	Objeto compartilhado entre Steps	Steps que passam dados entre si

Isolamento de dados

Cada cenário deve ser **independente**. Nunca assuma que um cenário anterior criou dados que o próximo vai usar. Estratégias para isolamento:

1. **API pública de teste (JSONPlaceholder):** Dados já existem, não precisam ser criados
2. **Setup no @Dado:** Cada cenário cria seus próprios dados
3. **Cleanup no @After:** Cada cenário limpa o que criou
4. **Dados únicos:** Use Faker para evitar colisão entre execuções paralelas

Exemplo de cenário auto-contido:

```
Cenário: Editar post existente
  Dado que criei um novo post      ← SETUP próprio
  Quando edito o título do post    ← ACAO
  Então o título deve estar atualizado ← VERIFICACAO
  # @After: deleta o post criado    ← CLEANUP automático
```

Dica: Se seus testes só passam em uma ordem específica, eles estão MAL escritos. Cada cenário deve passar isoladamente: `mvn test -Dcucumber.filter.tags="@cenário-especifico"`.

Apendice J — Depuracao Avancada

Tecnicas de debug para cada camada

1. Feature não executa:

```
# Verifique se o Cucumber encontra a feature
mvn test -Dcucumber.filter.tags="@sua-tag" -X 2>&1 | grep "feature"
```

2. Step não e encontrado: - Verifique se o texto no .feature bate EXATAMENTE com a anotacao - Verifique acentos e caracteres especiais - Verifique se o package esta no glue do Runner

3. Elemento não encontrado (UI):

```
// Adicione log temporario para debug:
LogUtils.debug("URL atual: " + driver.getCurrentUrl());
LogUtils.debug("Page source: " + driver.getPageSource().substring(0, 500));
```

4. Request falha (API): - Verifique o attachment no Allure (request completa) - Execute a mesma request no Postman manualmente - Verifique se a baseUrl esta correta

5. Teste passa local, falha no CI: - Compare timeouts (CI e mais lento) - Verifique resolução de tela (headless) - Verifique conectividade de rede (firewall corporativo?) - Compare versoes de Chrome (local vs CI)

Modo verbose do Maven

Para maxima visibilidade durante debug:

```
# Output detalhado do Maven
mvn test -X

# Output detalhado do Cucumber
mvn test -Dcucumber.plugin="pretty"

# Combinacao (usar com filtro para não poluir)
mvn test -Dcucumber.filter.tags="@debug" -X
```

Dica: Crie uma tag @debug temporaria para isolar o cenário problemático. Rode apenas ele com output verbose. Remova a tag antes de commitar.

Apendice K — Segurança e Credenciais

Hierarquia de segurança para credenciais

Nível	Onde armazenar	Quando usar	Segurança
1	Arquivo .properties (DEV)	Desenvolvimento local	Baixa
2	Variáveis de ambiente	CI/CD (GitHub Secrets)	Média
3	Secrets Manager (Vault/AWS)	Produção	Alta
4	Certificados mTLS	APIs críticas	Muito alta

GitHub Secrets no pipeline:

Nunca coloque credenciais no arquivo `testes.yml`. Use secrets:

```
- name: Executar testes
  run: mvn test
  env:
    CI: true
    USUARIO_ADMIN: ${ secrets.USUARIO_ADMIN }
    SENHA_ADMIN: ${ secrets.SENHA_ADMIN }
```

O `ConfigReader` já resolve variáveis de ambiente automaticamente:

```
Propriedade: usuario.admin
→ Busca env var: USUARIO_ADMIN
→ Se existe: usa o valor da env var
→ Se não existe: usa o valor do .properties
```

Regras de segurança: - NUNCA commite credenciais de produção no repositório - Adicione `*.properties` com credenciais reais ao `.gitignore` se necessário - Use credenciais de teste em ambientes de teste (OrangeHRM demo = admin/admin123) - Rotacione credenciais periodicamente - Revise secrets antes de cada deploy

Dica: Credenciais em `.properties` commitadas são aceitáveis APENAS para ambientes de demonstração pública (como o OrangeHRM Demo). Para sistemas internos, use SEMPRE secrets.

Apendice L — Comparacao de Ferramentas

Selenium vs Alternativas

Aspecto	Selenium	Cypress	Playwright
Linguagem	Java, Python, C#, JS	JavaScript only	Java, Python, JS, C#
Arquitetura	WebDriver (externo)	In-browser	CDP (externo)
Browsers	Chrome, Firefox, Edge, Safari	Chrome, Firefox, Edge	Chrome, Firefox, WebKit
Velocidade	Media	Rapida	Rapida
Estabilidade	Media (waits manuais)	Alta (auto-wait)	Alta (auto-wait)
Ecossistema	Maduro (15+ anos)	Jovem (5 anos)	Jovem (3 anos)
Java 8	Selenium 3.x	N/A	Playwright Java
Comunidade	Enorme	Grande	Crescendo

Por que Selenium neste framework: - Compatibilidade com Java 8 (requisito corporativo) - Ecossistema maduro com integração Allure - Integração nativa com Cucumber Java - Maior quantidade de material em portugues - Padrão de mercado brasileiro (vagas de emprego)

REST Assured vs Alternativas

Aspecto	REST Assured	HttpClient	OkHttp	Retrofit
Finalidade	Testes	Producao	Producao	Producao
API	Fluente (given/when/then)	Verbosa	Media	Declarativa
Validacoes	Built-in	Manual	Manual	Manual
JsonPath	Integrado	Precisa de lib	Precisa de lib	Precisa de lib

Schema validation	Plugin nativo	Manual	Manual	Manual
BDD style	Sim	Não	Não	Não

Dica: Escolha a ferramenta pelo *ECOSSISTEMA*, não pela *feature individual*. Um *framework* com boa documentação, comunidade ativa e integração com seu *stack* vale mais que uma ferramenta "melhor" mas isolada.

Apendice M — Maven em Profundidade

Ciclo de vida do Maven

O Maven organiza a construção em fases sequenciais. Cada fase executa um conjunto de plugins:

```
validate → compile → test-compile → test → package → verify → install → deploy
```

Quando você executa `mvn test`, o Maven executa TODAS as fases até `test` inclusive: 1. `validate` — valida se o projeto está correto 2. `compile` — compila código-fonte principal (`src/main`) 3. `test-compile` — compila código de teste (`src/test`) 4. `test` — executa testes via Surefire Plugin

Surefire Plugin:

O Surefire é o plugin que efetivamente roda os testes. Ele: - Procura classes que correspondem ao padrão de inclusão (`**/TestRunner.java`) - Cria uma JVM separada (fork) para execução - Injeta argumentos via `<argLine>` (AspectJ agent, truststore) - Define propriedades de sistema via `<systemPropertyVariables>` - Gera relatórios XML em `target/surefire-reports/`

Gerenciamento de dependências

O Maven resolve dependências transitivas automaticamente. Por exemplo:

```
selenium-java 3.141.59
■■■ guava 25.0-jre (transitiva)
■■■ okhttp 3.11.0 (transitiva)
■■■ commons-exec 1.3 (transitiva)
■■■ ...
```

Você declara `selenium-java` e o Maven baixa automaticamente tudo que o Selenium precisa.

Scopes importantes:

Scope	Significado	Exemplo
<code>compile</code> (padrão)	Disponível em todo lugar	SLF4J, Logback
<code>test</code>	Apenas em <code>src/test</code>	JUnit, Cucumber, REST Assured
<code>provided</code>	Fornecido pelo container	Servlet API
<code>runtime</code>	Apenas em execução	JDBC drivers

No nosso pom.xml, a maioria das dependências e `test` porque o framework inteiro vive em `src/test/`.

Profiles Maven (avancado)

Para cenários complexos, profiles permitem configuracoes condicionais:

```
<profiles>
  <profile>
    <id>smoke</id>
    <properties>
      <cucumber.filter.tags>@smoke</cucumber.filter.tags>
    </properties>
  </profile>
  <profile>
    <id>api-only</id>
    <properties>
      <cucumber.filter.tags>@api</cucumber.filter.tags>
    </properties>
  </profile>
</profiles>
```

Uso: `mvn test -Psmoke` (ativa o profile "smoke").

Dica: Profiles sao uteis quando você tem configuracoes complexas demais para a linha de comando. Para filtros simples de tag, `-Dcucumber.filter.tags` e suficiente.

Apendice N — Integração com Ferramentas de Gestão

Rastreabilidade com Jira/Azure DevOps

Para conectar testes automatizados com requisitos no backlog, use tags rastream:

```
@JIRA-1234
Cenário: Login com credenciais válidas
...
```

No Allure Report, essa tag aparece como link rastreável. Para ativar:

```
// Em um hook global:
@Before
public void linkToIssue(Scenário cenário) {
    for (String tag : cenário.getSourceTagNames()) {
        if (tag.startsWith("@JIRA-")) {
            Allure.link("Jira", "https://jira.empresa.com/browse/" + tag.replace("@", ""));
        }
    }
}
```

Exportação de resultados

Os resultados podem ser exportados para diversas ferramentas:

Formato	Arquivo	Ferramenta destino
JUnit XML	target/cucumber-reports/cucumber-junit.xml	Jenkins, Azure DevOps, GitHub
JSON	target/cucumber-reports/cucumber-embed.json	Cucumber Reports, Xray
Allure Results	target/allure-results/	Allure Server, Allure TestOps
HTML	target/cucumber-reports/cucumber-embed.html	qualquer navegador

Para Xray (Jira plugin): O arquivo `cucumber.json` pode ser importado diretamente no Xray para atualizar status de execução de testes vinculados a histórias.

Para TestRail: Existem plugins que convertem `cucumber.json` para o formato de importação do TestRail.

Dica: A rastreabilidade bidirecional (requisito → teste → resultado) é exigida em auditorias de qualidade (ISO, SOX). Invista nisso desde o início.

Apendice O — Execução Paralela

Como configurar paralelismo

O framework já está preparado para execução paralela graças ao `ThreadLocal` no `DriverManager`. Para ativar:

Opcao 1: Surefire com fork paralelo

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <forkCount>2</forkCount>
    <reuseForks>true</reuseForks>
  </configuration>
</plugin>
```

Opcao 2: Cucumber paralelo nativo (com plugin)

```
<plugin>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit-platform-engine</artifactId>
  <configuration>
    <parallel>true</parallel>
    <threadCount>4</threadCount>
  </configuration>
</plugin>
```

Pre-requisitos para paralelismo: - [x] `DriverManager` com `ThreadLocal` (já implementado) - [x] `PicoContainer` com escopo por cenário (já implementado) - [x] Cenários independentes (sem compartilhar estado) - [] Dados de teste isolados (nenhum cenário altera dados de outro) - [] Infraestrutura suporta múltiplos browsers simultâneos

Ganho de performance:

Cenários	Serial	2 threads	4 threads
10 UI (5s cada)	50s	25s	13s
20 API (1s cada)	20s	10s	5s
Total estimado	70s	35s	18s

Dica: Comece com paralelismo apenas nos testes de API (sem browser, sem conflito visual). Adicione UI paralelo depois, com cuidado.

Apendice P — Padroes de Assercao

Assercoes claras e informativas

A qualidade da mensagem de erro determina quanto tempo você gasta debugando. Compare:

```
// RUIM - mensagem generica
Assert.assertTrue(loginPage.isOnDashboard());
// Erro: "expected true but was false" – INUTIL

// BOM - mensagem descritiva
Assert.assertTrue("Não redirecionou para o dashboard", loginPage.isOnDashboard());
// Erro: "Não redirecionou para o dashboard" – CLARO
```

Padrão de assercoes por tipo

Tipo de validação	Metodo	Quando usar
Igualdade exata	<code>assertEquals(expected, actual)</code>	Valores conhecidos
Verdadeiro/Falso	<code>assertTrue(msg, condition)</code>	Estados booleanos
Não nulo	<code>assertNotNull(msg, value)</code>	Campos obrigatórios
Diferente	<code>assertNotEquals(msg, unexpected, actual)</code>	Campos nao-vazio
Contem texto	<code>assertTrue(msg, text.contains(sub))</code>	Textos parciais
Lista vazia	<code>assertFalse(msg, list.isEmpty())</code>	Arrays com dados

Assercoes compostas (Soft Assertions)

Para cenários que precisam validar MULTIPLOS campos sem parar no primeiro erro:

```
// Com JUnit padrão: para no PRIMEIRO erro
Assert.assertEquals("userId incorreto", 1, response.jsonPath().getInt("userId"));
Assert.assertEquals("id incorreto", 1, response.jsonPath().getInt("id"));
```

```
// Se userId falha, id nunca e verificado  
// Com Soft Assert (AssertJ ou custom): verifica TUDO  
SoftAssertions soft = new SoftAssertions();  
soft.assertThat(response.jsonPath().getInt("userId")).isEqualTo(1);  
soft.assertThat(response.jsonPath().getInt("id")).isEqualTo(1);  
soft.assertAll(); // Reporta TODAS as falhas de uma vez
```

O framework atual usa JUnit padrão (para no primeiro erro). Para cenários complexos com muitas validacoes, considere adicionar AssertJ como dependência futura.

Dica: SEMPRE inclua uma mensagem descritiva em `Assert`. O custo e minimo (uma `String`) é o beneficio e enorme quando o teste falha as 3 da manha no CI.

Apendice Q — Organização de Features por Dominio

Estrutura recomendada para projetos grandes

Para projetos com 50+ cenários, organize features por dominio de negocio:

```
resources/features/  
  ui/  
    autenticacao/  
      login.feature  
      logout.feature  
      recuperar-senha.feature  
    cadastro/  
      cadastro-usuario.feature  
      cadastro-empresa.feature  
    dashboard/  
      dashboard.feature  
  api/  
    posts/  
      posts.feature  
    users/  
      users.feature  
    comments/  
      comments.feature  
  integração/  
    login-api-ui.feature
```

Tags compostas para filtros complexos

```
@ui @autenticacao @smoke @sprint-15 @JIRA-2345  
Cenário: Login com credenciais válidas
```

Filtros uteis:

```
# Tudo da sprint atual  
mvn test -Dcucumber.filter.tags="@sprint-15"  
  
# Smoke do dominio de autenticacao  
mvn test -Dcucumber.filter.tags="@autenticacao and @smoke"  
  
# Regressao completa exceto sprint atual (ainda instavel)  
mvn test -Dcucumber.filter.tags="not @sprint-15"
```

Dica: Não exagere nas tags. 3-4 tags por cenário é o maximo razoavel. Tags demais poluem e ninguem lembra de usa-las.

Apêndice R — Monitoramento e Alertas

Configurando notificações de falha

Quando o pipeline falha, o time deve ser notificado imediatamente. Opções:

1. GitHub Actions + Slack:

```
- name: Notificar falha no Slack
  if: failure()
  uses: 8398a7/action-slack@v3
  with:
    status: ${ job.status }
    channel: '#qa-automação'
  env:
    SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK }
```

2. GitHub Actions + Email: O GitHub já envia emails automáticos para o autor do commit quando o pipeline falha.

3. GitHub Actions + Teams:

```
- name: Notificar falha no Teams
  if: failure()
  uses: jdcargile/ms-teams-notification@v1.4
  with:
    github-token: ${ secrets.GITHUB_TOKEN }
    ms-teams-webhook-uri: ${ secrets.TEAMS_WEBHOOK }
```

Dashboard de saúde

Crie uma página com badges de status no README do repositório:

```
![CI Status](https://github.com/{org}/{repo}/actions/workflows/testes.yml/badge.svg)
![Allure Report](https://img.shields.io/badge/Report-Allure-orange)
```

Isso dá visibilidade instantânea ao time sobre a saúde dos testes.

Dica: Defina um SLA para falhas de teste: "toda falha de smoke deve ser investigada em até 4 horas úteis". Sem SLA, alertas viram ruído.

Consideracoes Finais

Este framework representa uma base profissional para automação de testes que equilibra simplicidade de uso com robustez arquitetural. Cada decisão — desde a escolha do Java 8 ate a estratégia de screenshots configuraveis — foi tomada pensando em cenários reais de empresas brasileiras.

O mais importante não e decorar cada classe, mas entender os PRINCÍPIOS por tras delas:

1. **Separação de responsabilidades** — cada classe faz UMA coisa bem
2. **Configuração externalizada** — nenhum valor fixo no código
3. **Injecao de dependência** — objetos sao recebidos, não criados
4. **Reusabilidade** — steps genericos, Pages herdam de base
5. **Observabilidade** — logs, screenshots e reports em toda execução

Com esses principios internalizados, você consegue construir e MANTER frameworks profissionais em qualquer tecnologia.

O proximo passo e seu: aplique esses conceitos em um projeto real, adapte as convencoes ao contexto da sua empresa, e itere continuamente. Um framework nunca esta "pronto" — ele evolui junto com o time é o produto.

Apendice S — Exercícios Práticos

Para fixar os conceitos, realize estes exercícios progressivos:

Exercício 1: Novo Page Object (Básico)

Objetivo: Criar um Page Object para a tela de Dashboard (após login bem-sucedido).

Tarefas: 1. Crie `pages/dashboard/DashboardPage.java` 2. Estenda `BasePage` 3. Identifique 3 elementos na página (menu lateral, header, username) 4. Crie métodos: `getLoggedUsername()`, `clickMenuItem(String)`, `isMenuVisible()` 5. Crie cenário na feature: "Então devo ver o username 'Admin' no header"

Validação: O teste deve passar com `mvn test -Dcucumber.filter.tags="@dashboard"`

Exercício 2: Novo Endpoint API (Intermediário)

Objetivo: Adicionar testes para o endpoint `/comments` da JSONPlaceholder API.

Tarefas: 1. Crie `api/models/CommentRequest.java` (name, email, body, postId) 2. Crie `api/services/CommentService.java` (listAll, getByPost, create) 3. Crie `payloads/comments/create-comment.json` 4. Crie `schemas/comment-schema.json` 5. Crie `features/api/comments.feature` com 5 cenários 6. Crie `steps/api/CommentSteps.java`

Validação: `mvn test -Dcucumber.filter.tags="@api"` deve passar com os novos cenários

Exercício 3: Multi-ambiente (Avançado)

Objetivo: Configurar um terceiro ambiente e executar testes contra ele.

Tarefas: 1. Crie `environments/staging.properties` com URLs diferentes 2. Adicione timeouts mais generosos (25s explicit) 3. Execute: `mvn test -Denvironment=staging -Dcucumber.filter.tags="@smoke"` 4. Verifique nos logs que o ambiente correto foi carregado 5. Compare relatórios de execução entre ambientes

Exercício 4: CI/CD (Avançado)

Objetivo: Criar um segundo workflow para execução agendada.

Tarefas: 1. Crie `.github/workflows/regressao-noturna.yml` 2. Configure trigger schedule: `cron('03 * * *')` (3h da manhã) 3. Execute todos os testes (sem filtro de tag) 4. Adicione notificação Slack/Teams em caso de falha 5. Publique resultados em branch separada do Pages

Validação: O workflow deve aparecer na aba Actions do GitHub e executar no horario configurado.

Exercicio 5: Refatoracao (Expert)

Objetivo: Extrair steps genericos reusaveis.

Tarefas: 1. Identifique steps que se repetem entre PostSteps e futuros CommentSteps 2. Crie `steps/common/CommonApiSteps.java` 3. Mova validacoes genericas (status code, content-type, schema, campo) 4. Ajuste PostSteps para herdar apenas steps especificos 5. Verifique que todos os testes continuam passando

Validação: `mvn test` deve passar sem nenhuma alteracao nas features

Apendice T — Referencias e Leitura Complementar

Livros recomendados

Livro	Autor	Topico
Clean Code	Robert C. Martin	Principios de código limpo
Design Patterns	Gang of Four	Padroes de projeto
Continuous Delivery	Jez Humble	CI/CD e DevOps
Agile Testing	Lisa Crispin	Testes em contexto agil
BDD in Action	John Ferguson Smart	Cucumber e BDD
Java Efetivo	Joshua Bloch	Boas praticas Java

Documentacoes oficiais

Ferramenta	URL
Selenium	https://www.selenium.dev/documentation/
Cucumber	https://cucumber.io/docs/cucumber/
REST Assured	https://rest-assured.io/
Allure	https://docs.qameta.io/allure/
Maven	https://maven.apache.org/guides/
GitHub Actions	https://docs.github.com/en/actions
JUnit 4	https://junit.org/junit4/
SLF4J	https://www.slf4j.org/manual.html
JavaFaker	https://github.com/DiUS/java-faker

Comunidades

- Stack Overflow (tags: selenium, cucumber, rest-assured)
- GitHub Discussions dos projetos oficiais
- Grupos de QA no LinkedIn Brasil
- Canal Selenium Brazil no Telegram

Dica: A documentação oficial e *SEMPRE* a fonte mais confiável. Blogs e tutoriais podem estar desatualizados. Quando em duvida, consulte a doc oficial primeiro.

Nota Final do Autor

Este material foi construído com base em experiência real de projetos de automação em empresas brasileiras de médio e grande porte. Cada padrão apresentado aqui foi validado em produção com suites de 200+ cenários e times de 3-10 QAs.

A automação de testes não é apenas uma habilidade técnica — é uma mentalidade. O QA que domina automação não substitui o teste manual; ele AMPLIFICA sua capacidade de garantir qualidade. Com um framework sólido, você pode:

- Executar 100 cenários de regressão em 10 minutos (o que levaria 2 dias manual)
- Detectar regressões em minutos após cada deploy
- Gerar evidências automáticas para auditoria
- Liberar tempo para testes exploratórios e de usabilidade

O investimento vale a pena. O retorno é mensurável. E o mercado paga mais por profissionais que dominam essas técnicas.

Bons testes!